

Sequence Containers

Bad Dad Joke of the Day:

- Two dragons are in a bar. One dragon says, “It’s warm in here.” The other replies, “Shut your mouth.”

Creds: EW

Game Plan



- Finishing Up C++ Types
- Survey Results!
- Overview of STL
- Sequence Containers
- Container Adaptors

C++ Types (cont.)

structures

C++17 allows structured bindings, allowing you to unpack the variables in a pair.

```
pair<int, int> findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return make_pair(min, max);  
}
```

```
int main() {  
    int dist = 6452;  
    auto [min, max] = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << min << " and " << max;  
}
```

Notice the use of auto here – automatically infer the type of each variable.

C++17 allows structured bindings, allowing you to unpack the variables in a pair.

```
pair<int, int> findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return make_pair(min, max);  
}
```

```
int main() {  
    int dist = 6452;  
    auto [min, max] = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << min << " and " << max;  
}
```

What drawbacks of this approach can you see?

A struct is a collection of named variables grouped together.

```
struct PriceRange {  
    int min;  
    int max;  
}
```

struct PriceRange

int min	650
int max	1729

```
struct Course {  
    string code;  
    Time startTime; Time endTime;  
  
    vector<string> instructors;  
}
```

struct Course

string code	"CS106L"
Time startTime	15:30
Time endTime	16:20
vector<string> instructors;	

A struct is a collection of named variables grouped together.

```
struct PriceRange {  
    int min;  
    int max;  
}
```

```
struct Course {  
    string code;  
    Time startTime; Time endTime;  
  
    vector<string> instructors;  
}
```

Same as a `pair<int, int>`, but the ints are named.

Structs offer the benefit that the struct itself and the variables inside are named.

```
PriceRange findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return PriceRange{min, max};  
}
```

```
int main() {  
    int dist = 6452;  
    PriceRange p = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << p.min << " and " << p.max;  
}
```

This is very readable: result is a DatingRange, and you are printing its min and max.

Structs offer the benefit that the struct itself and the variables inside are named.

```
PriceRange findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return PriceRange{min, max};  
}
```

```
int main() {  
    int dist = 6452;  
    PriceRange p = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << p.min << " and " << p.max;  
}
```

To access a member inside the struct, use the . notation.

You can also use structured bindings on structs.

```
PriceRange findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return PriceRange{min, max};  
}
```

```
int main() {  
    int dist = 6452;  
    auto [min, max] = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << min << " and " << max;  
}
```

The order the binding occurs is the same order as the variables are laid in the struct.

Structures are frequently used in practice.

```
std::tuple<bool, Time, Time> findCourseTime([omitted]) {  
    for (int i = 0; i < courseDatabase.size(); ++i) {  
        if (courseCode == course.code) {  
            return make_tuple(true, course.startTime,  
                               course.endTime);  
        }  
    }  
  
    return make_tuple(false, Time{}, Time{});  
}
```

Note: we'll clean this up using uniform initialization later!

We want to go through all the Courses in the vector and add 1 to start and end time.

```
void transformToDST(vector<Course>& courses) {  
    for (auto& course : courses) {  
        course.startTime.hour++;  
        course.endTime.hour++;  
    }  
}
```

Remember: auto discards all
qualifiers.

Can also use structured binding with references!

```
void transformToDST(vector<Course>& courses) {  
    for (auto& [code, start, end, instructors] : courses) {  
        start++;  
        end++;  
    }  
}
```

In each iteration, unpacks
each member as a reference.

Review from 106B: use const on large parameters.

```
void transformToDST(const vector<Course>& courses) {  
    for (auto& course : courses) {  
        course.startTime.hour++;  
        course.endTime.hour++;  
    }  
}
```

Use const on local variables that is not meant to change (especially references).

```
void transformToDST(const vector<Course>& courses) {  
    for (const auto& course : courses) {  
        course.startTime.hour++;  
        course.endTime.hour++;  
    }  
}
```

initialization

In C++, depending on the type, there were too many ways to initialize a variable.



To solve this, C++11 (ironically) adds one more way: uniform initialization.

```
int main() {  
    vector<int> vec{3, 1, 4, 1, 5, 9};  
    Course now {"CS106L", {15, 30}, {16, 30},  
               {"Wang", "Zeng"} };  
}
```

We don't have to specify the types – automatically deduced.

The return value can also be uniform initialized.

```
pair<int, int> findPriceRange(int dist) {  
    int min = static_cast<int>(dist * 0.08 + 100);  
    int max = static_cast<int>(dist * 0.36 + 750);  
    return {min, max};  
}
```

```
int main() {  
    int dist = 6452;  
    auto [min, max] = findPriceRange(dist);  
    cout << "You can find prices between: "  
        << min << " and " << max;  
}
```


A initializer list is a lightweight vector that can be used as a parameter.

```
vector::vector(initializer_list<T> init);
```

Constructor creates a vector
with initial elements.

```
vector<int> vec{3, 1, 4, 1, 5, 9};
```

Using the uniform initialization syntax, the initializer list ctor is preferred over constructor.

```
int main() {  
    vector<int> vec1{3}; // vector = {3}  
    vector<int> vec2(3); // vector = {0, 0, 0}  
}
```

First one calls ctor with initializer list, second calls constructor with one parameter.

Caution: Use Thoughtfully

When should I use a stringstream?

- Processing strings
 - Simplify `“/./a/b/..”` to `“/a”`
- Formatting input/output
 - `uppercase`, `hex`, and other stream manipulators
- Parsing different types
 - `stringToInteger()` from previous lectures



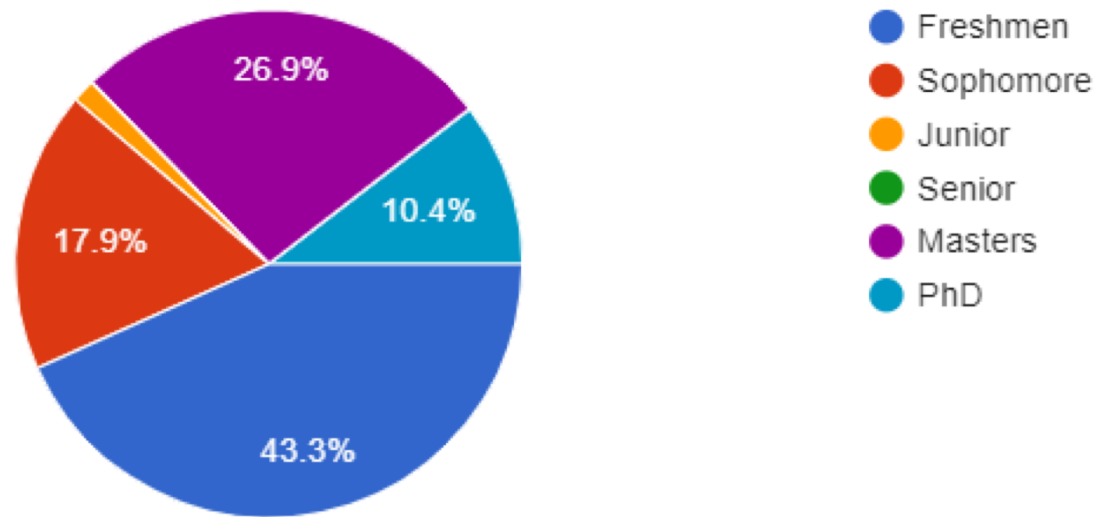
Caution: Use Thoughtfully

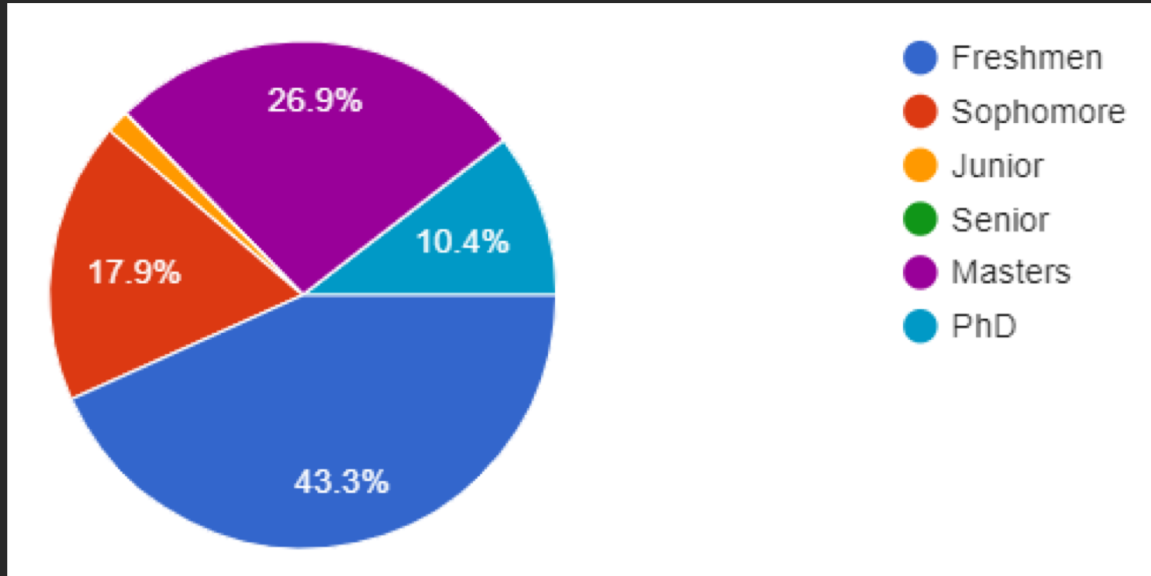
When should I use a stringstream?

- Processing strings
 - Simplify `“/./a/b/..”` to `“/a”`
- Formatting input/output
 - `uppercase`, `hex`, and other stream manipulators
- Parsing different types
 - `stringToInteger()` from previous lectures

If you're just concatenating strings, `str.append()` is faster than using a stringstream!

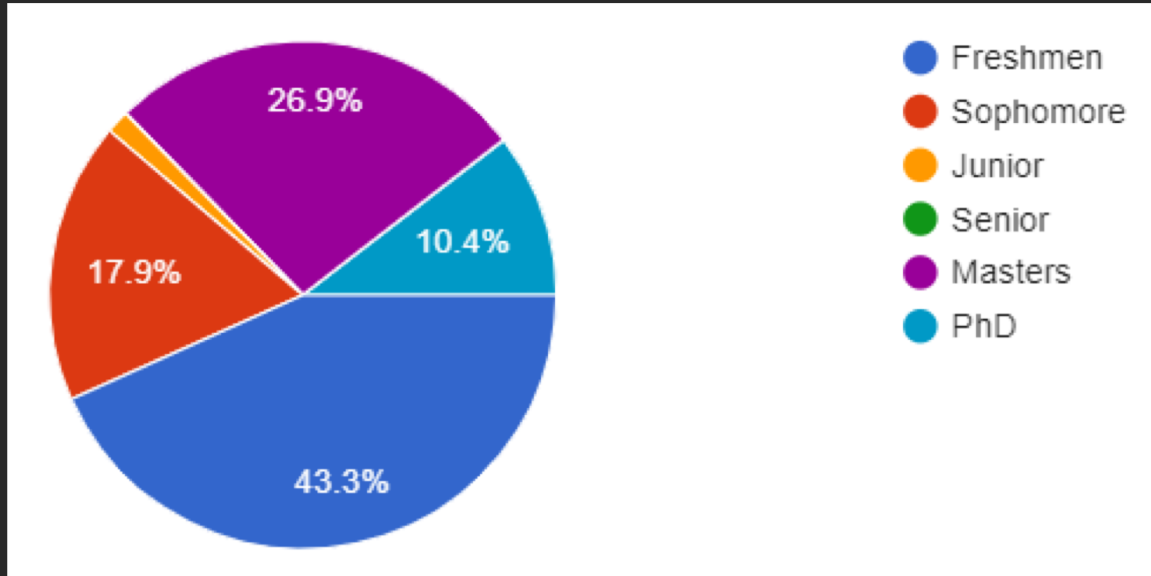
Survey Results!





Majors:

- Computer Science
- Aero/Astro
- Linguistics
- Materials Science
- Mechanical Engineering
- Public Policy

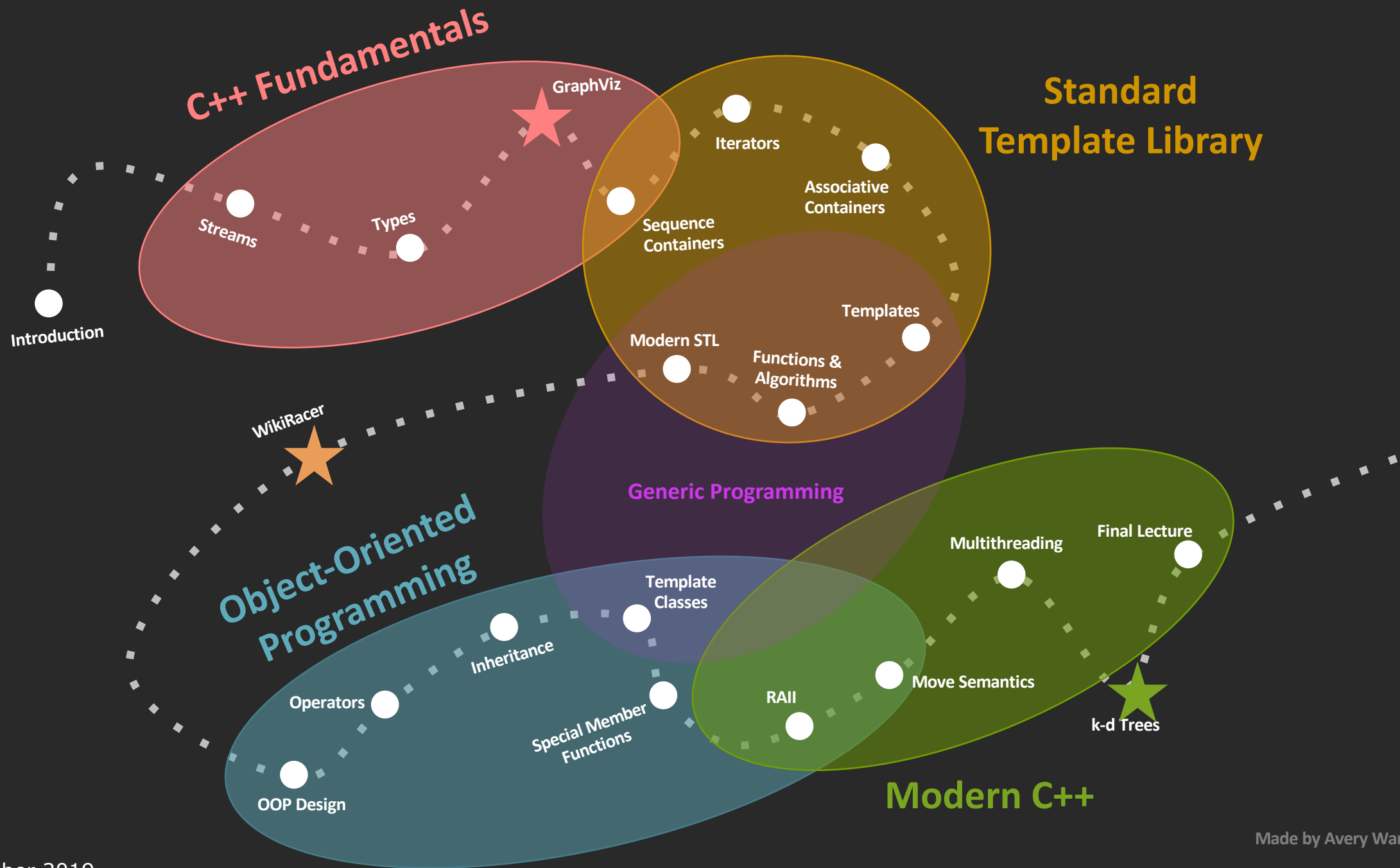


Why you're here:

- Industry usages
- C++ practice
- Supplement CS 106B/X
- Specific applications

Majors:

- Computer Science
- Aero/Astro
- Linguistics
- Materials Science
- Mechanical Engineering
- Public Policy



Overview of STL

Overview of STL

“As mathematicians learned to lift theorems into their most **general** setting, so I wanted to lift **algorithms and data structures**.”

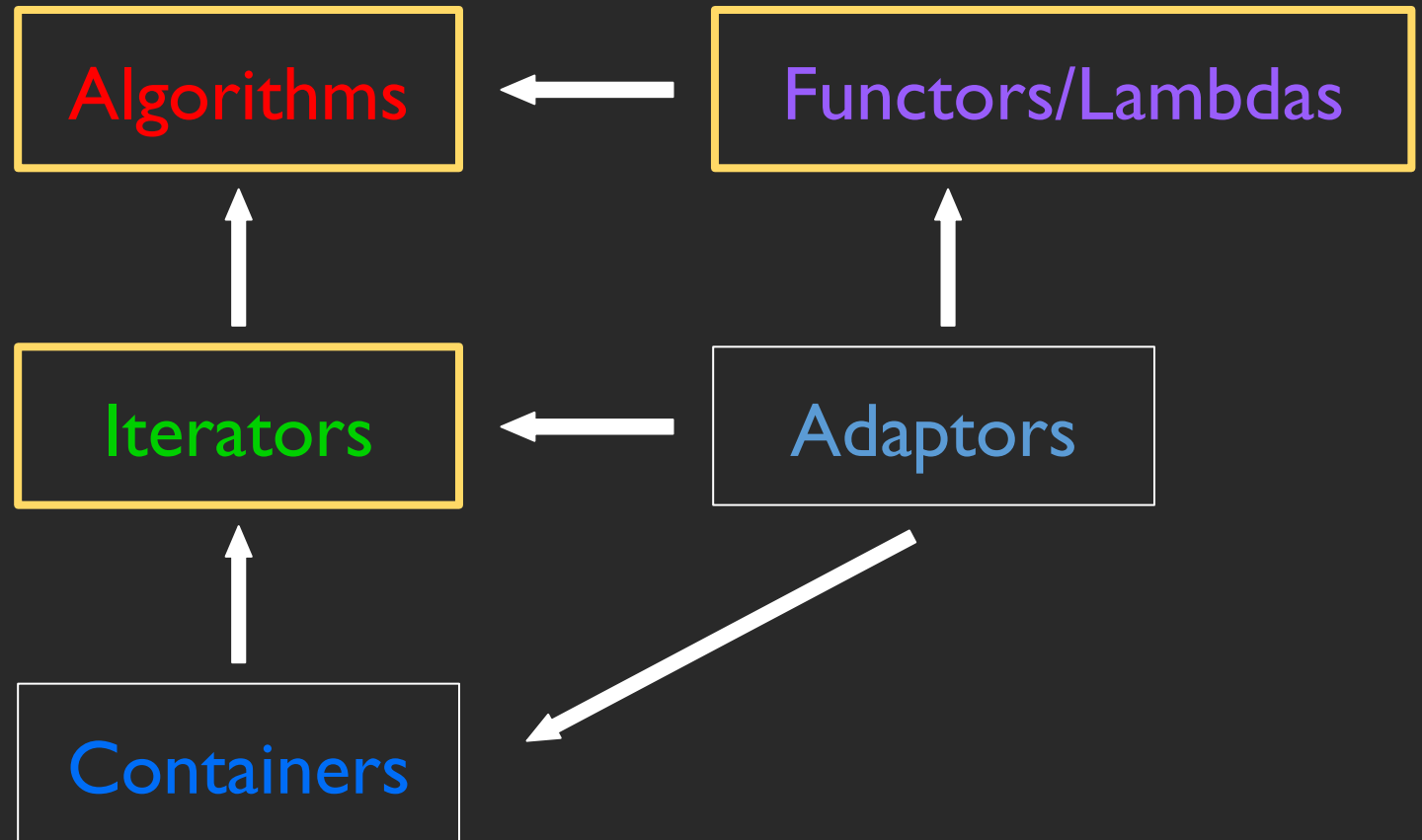
— *Alex Stepanov, inventor of the STL*



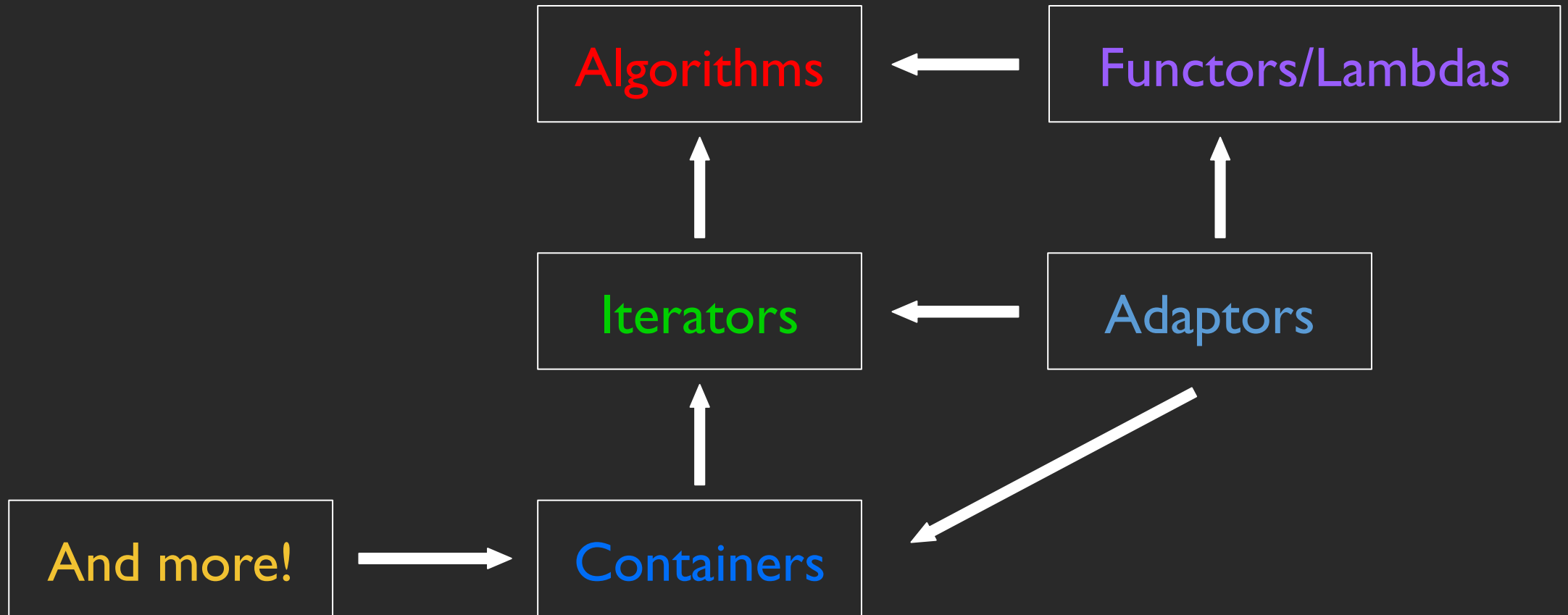
Overview of STL

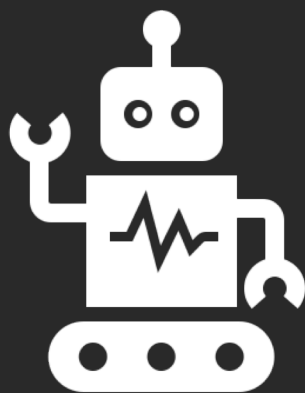


Overview of STL



Overview of STL





Example

The Power of the C++ STL

Where we are going...

Here is a program that generates a vector with random entries, sorts it, and prints it, all in one go!

```
const int kNumInts = 200;
std::vector<int> vec(kNumInts);
std::generate(vec.begin(), vec.end(), rand);
std::sort(vec.begin(), vec.end());
std::copy(vec.begin(), vec.end(),
          std::ostream_iterator<int>(cout, "\n"));
```


Overview of STL

“Another benefit of striving for efficiency is that the process forces you to understand the problem in more depth.”

— *also Alex Stepanov, inventor of the STL*



Sequence Containers

Sequence Containers

Provides access to **sequences** of elements.

Includes:

- `std::vector<T>`
- `std::deque<T>`
- `std::list<T>`
- `std::array<T>`
- `std::forward_list<T>`

```
std::vector<T>
```

`std::vector<T>`

A vector represents a sequence of elements of **any** type.

You specify the type when using the vector:

```
std::vector<int> vecInt;           // vector of ints
std::vector<string> vecStr;        // vector of string
std::vector<myStruct> vecStruct;    // vector of myStructs
std::vector<std::vector<string>> vecOfVec; // vector of
                                         vector<string>
```

Summary of Stanford `Vector<T>` vs `std::vector<T>`

What you want to do	Stanford <code>Vector<int></code>	<code>std::vector<int></code>
Create an empty vector	<code>Vector<int> v;</code>	<code>vector<int> v;</code>
Create a vector with n copies of zero	<code>Vector<int> v(n);</code>	<code>vector<int> v(n);</code>
Create a vector with n copies of a value k	<code>Vector<int> v(n, k);</code>	<code>vector<int> v(n, k);</code>
Add k to the end of the vector	<code>v.add(k);</code>	<code>v.push_back(k);</code>
Clear vector	<code>v.clear();</code>	<code>v.clear();</code>
Get the element at index i (verify that i is in bounds)	<code>int k = v.get(i);</code> <code>int k = v[i];</code>	<code>int k = v.at(i);</code> <code>int k = v[i];</code>
Check if the vector is empty	<code>if (v.isEmpty()) ...</code>	<code>if (v.empty()) ...</code>
Replace the element at index i (verify that i is in bounds)	<code>v.get(i) = k;</code> <code>v[i] = k;</code>	<code>v.at(i) = k;</code> <code>v[i] = k;</code>

Summary of Stanford `Vector<T>` vs `std::vector<T>`

What you want to do	Stanford <code>Vector<int></code>	<code>std::vector<int></code>
Create an empty vector	<code>Vector<int> v;</code>	<code>vector<int> v;</code>
Create a vector with n copies of zero	<code>Vector<int> v(n);</code>	<code>vector<int> v(n);</code>
Create a vector with n copies of a value k	<code>Vector<int> v(n, k);</code>	<code>vector<int> v(n, k);</code>
Add k to the end of the vector	<code>v.add(k);</code>	<code>v.push_back(k);</code>
Clear vector	<code>v.clear();</code>	<code>v.clear();</code>
Get the element at index i (verify that i is in bounds)	<code>int k = v.get(i);</code> <code>int k = v[i];</code>	<code>int k = v.at(i);</code> <code>int k = v[i];</code>
Check if the vector is empty	<code>if (v.isEmpty()) ...</code>	<code>if (v.empty()) ...</code>
Replace the element at index i (verify that i is in bounds)	<code>v.get(i) = k;</code> <code>v[i] = k;</code>	<code>v.at(i) = k;</code> <code>v[i] = k;</code>

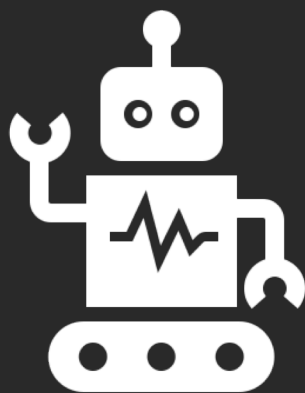
Summary of Stanford `Vector<T>` vs `std::vector<T>`

What you want to do	Stanford <code>Vector<int></code>	<code>std::vector<int></code>
Create an empty vector	<code>Vector<int> v;</code>	<code>vector<int> v;</code>
Create a vector with n copies of zero	<code>Vector<int> v(n);</code>	<code>vector<int> v(n);</code>
Create a vector with n copies of a value k	<code>Vector<int> v(n, k);</code>	<code>vector<int> v(n, k);</code>
Add k to the end of the vector	<code>v.add(k);</code>	<code>v.push_back(k);</code>
Clear vector	<code>v.clear();</code>	<code>v.clear();</code>
Get the element at index i (verify that i is in bounds)	<code>int k = v.get(i);</code> <code>int k = v[i];</code>	<code>int k = v.at(i);</code> <code>int k = v[i];</code>
Check if the vector is empty	<code>if (v.isEmpty()) ...</code>	<code>if (v.empty()) ...</code>
Replace the element at index i (verify that i is in bounds)	<code>v.get(i) = k;</code> <code>v[i] = k;</code>	<code>v.at(i) = k;</code> <code>v[i] = k;</code>

Summary of Stanford `Vector<T>` vs `std::vector<T>`

What you want to do	Stanford <code>Vector<int></code>	<code>std::vector<int></code>
Create an empty vector	<code>Vector<int> v;</code>	<code>vector<int> v;</code>
Create a vector with n copies of zero	<code>Vector<int> v(n);</code>	<code>vector<int> v(n);</code>
Create a vector with n copies of a value k	<code>Vector<int> v(n, k);</code>	<code>vector<int> v(n, k);</code>
Add k to the end of the vector	<code>v.add(k);</code>	<code>v.push_back(k);</code>
Clear vector	<code>v.clear();</code>	<code>v.clear();</code>
Get the element at index i (verify that i is in bounds)	<code>int k = v.get(i);</code> <code>int k = v[i];</code>	<code>int k = v.at(i);</code> <code>int k = v[i];</code>
Check if the vector is empty	<code>if (v.isEmpty()) ...</code>	<code>if (v.empty()) ...</code>
Replace the element at index i (verify that i is in bounds)	<code>v.get(i) = k;</code> <code>v[i] = k;</code>	<code>v.at(i) = k;</code> <code>v[i] = k;</code>

But there is one key difference!



Example

Standard C++ Vector in (Basic) Action

Why the Difference?

Why doesn't `std::vector` bounds check by default?

Hint: Remember our discussion of the philosophy of C++

Why the Difference?

Why doesn't `std::vector` bounds check by default?

Hint: Remember our discussion of the philosophy of C++

If you write your program **correctly**, bounds checking will just **slow** your code down.

One Important Similarity

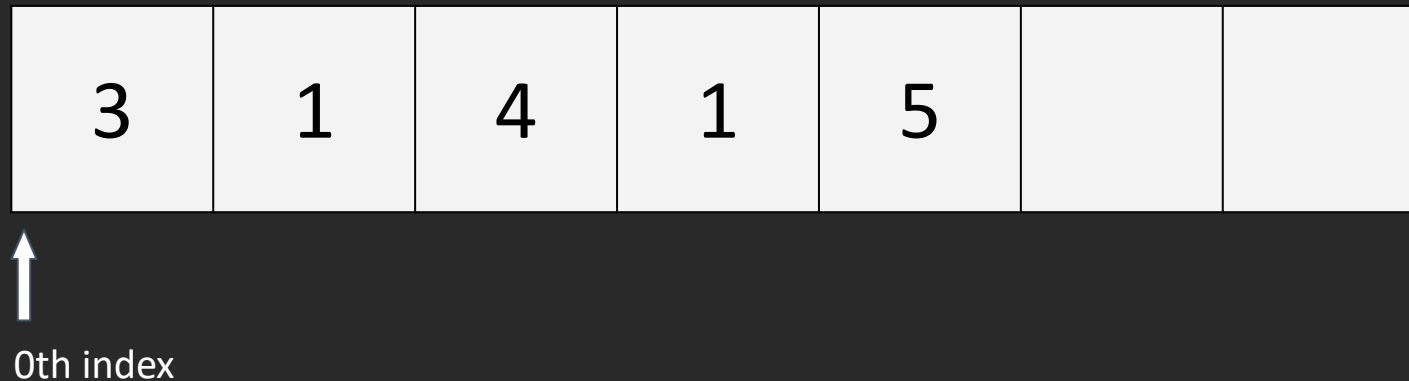
What you want to do	Stanford Vector<int>	std::vector<int>
Create an empty vector	<code>Vector<int> v;</code>	<code>vector<int> v;</code>
Create a vector with n copies of zero	<code>Vector<int> v(n);</code>	<code>vector<int> v(n);</code>
Create a vector with n copies of a value k	<code>Vector<int> v(n, k);</code>	<code>vector<int> v(n, k);</code>
Add k to the end of the vector	<code>v.add(k);</code>	<code>v.push_back(k);</code>
Clear vector	<code>v.clear();</code>	<code>v.clear();</code>
Get the bounds		
Check if		
Replace (within bounds)	<code>v[i] = k;</code>	<code>v[i] = k;</code>

What happens if we try to add an element to the **beginning** of a vector?

Why is `push_front` slow?

Why is `push_front` slow?

Suppose `push_front` existed and we used it.
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

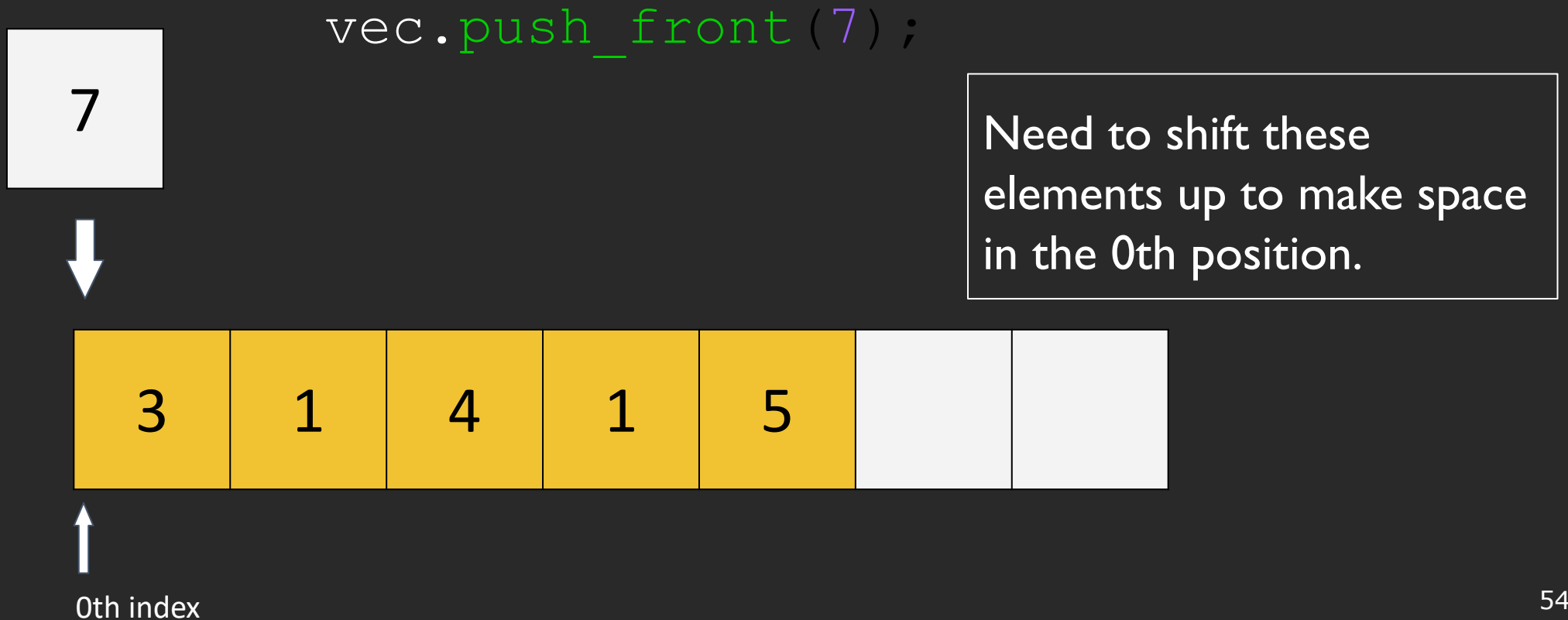
Let's look at a small vector:



Why is `push_front` slow?

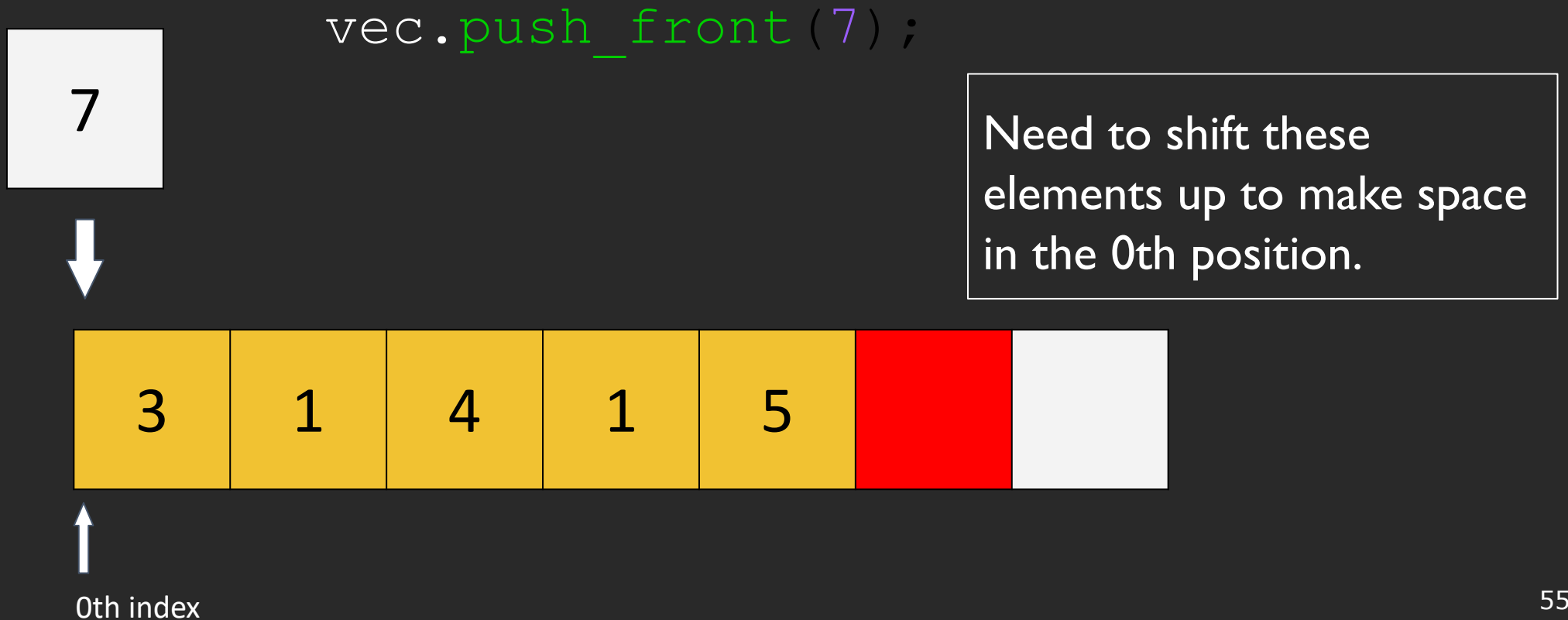
Suppose `push_front` existed and we used it.

Let's look at a small vector:



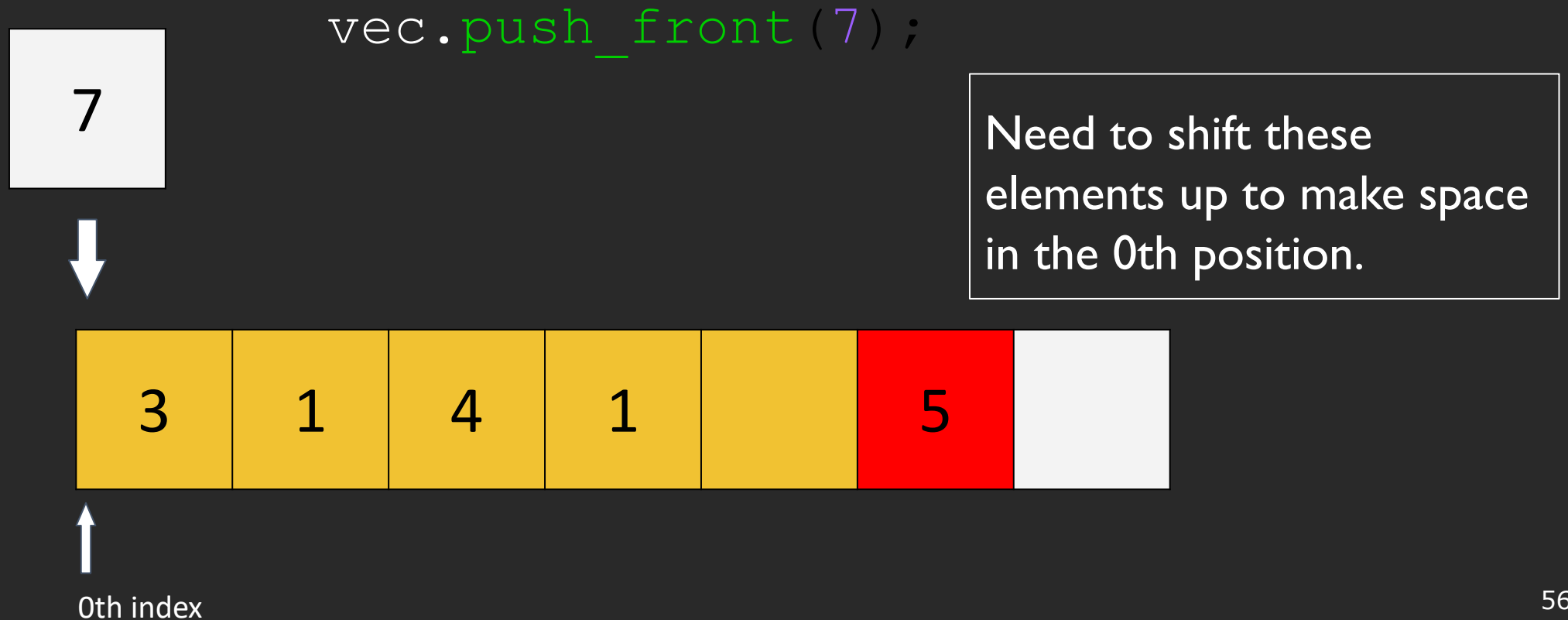
Why is `push_front` slow?

Suppose `push_front` existed and we used it.
Let's look at a small vector:



Why is `push_front` slow?

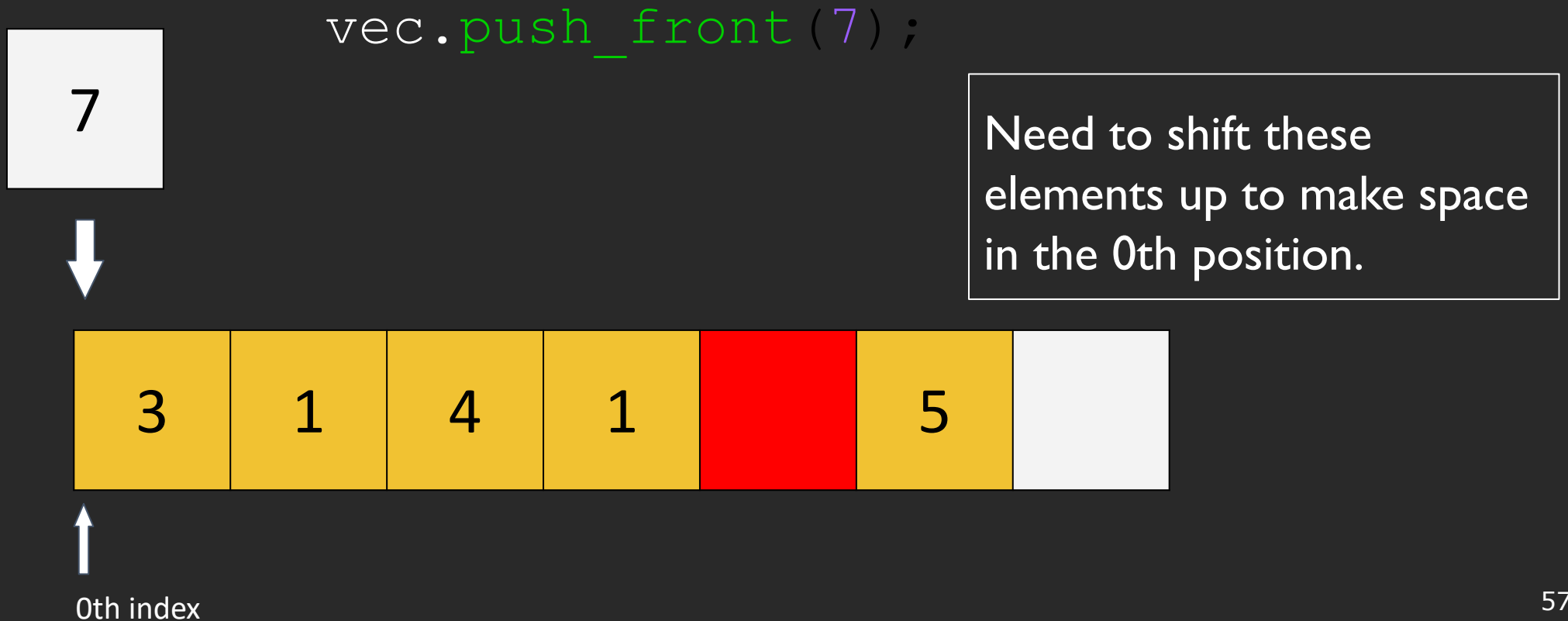
Suppose `push_front` existed and we used it.
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

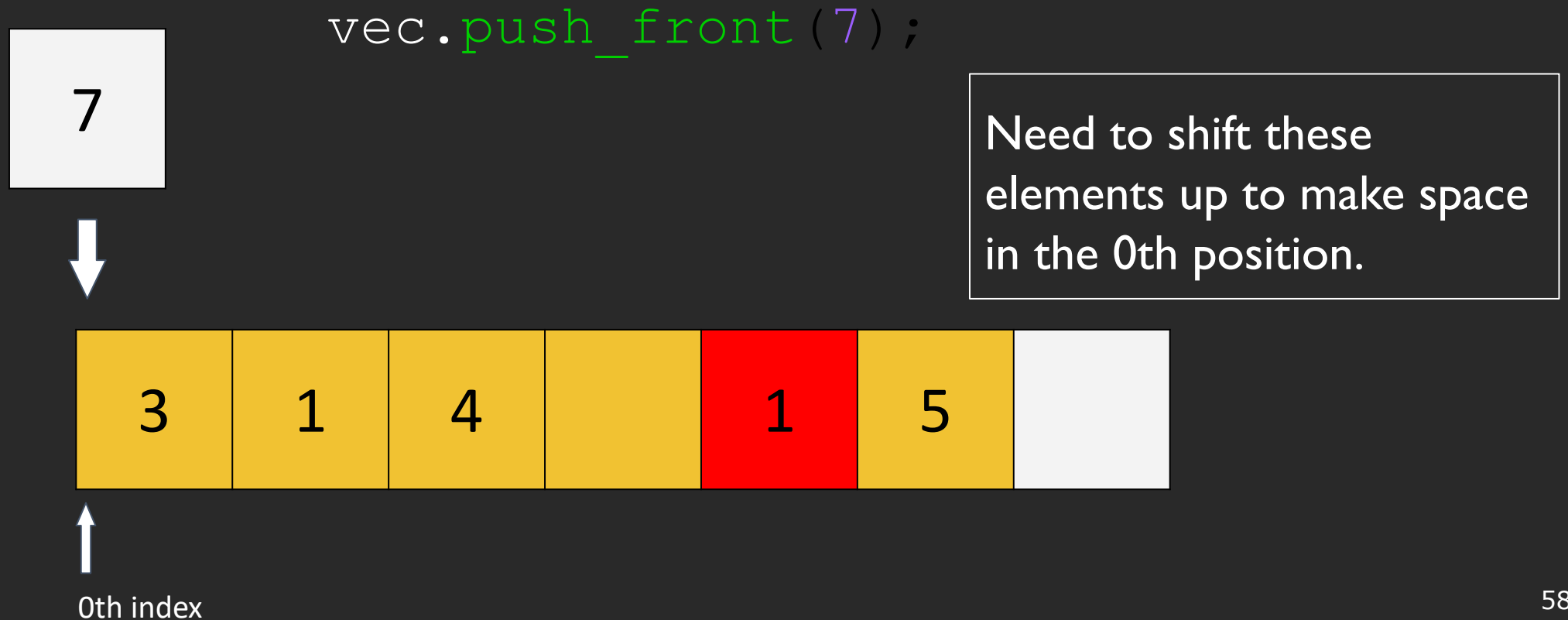
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

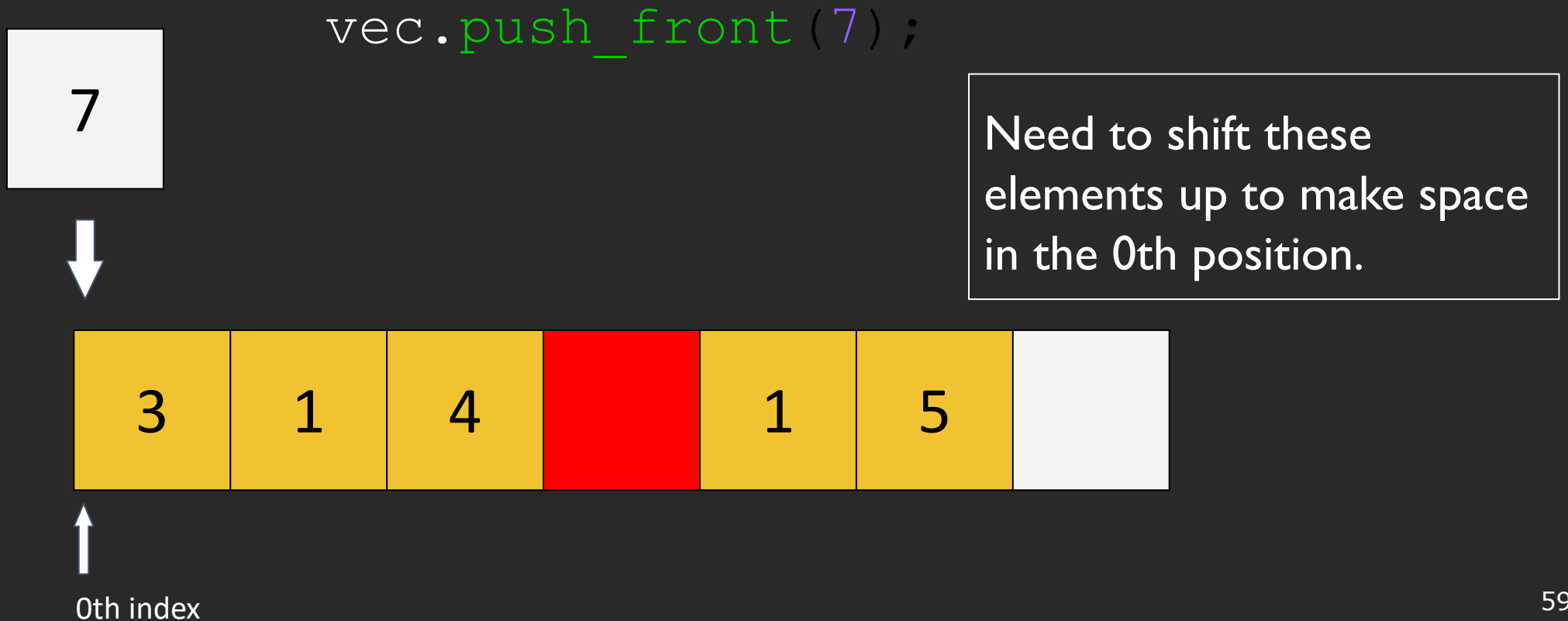
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

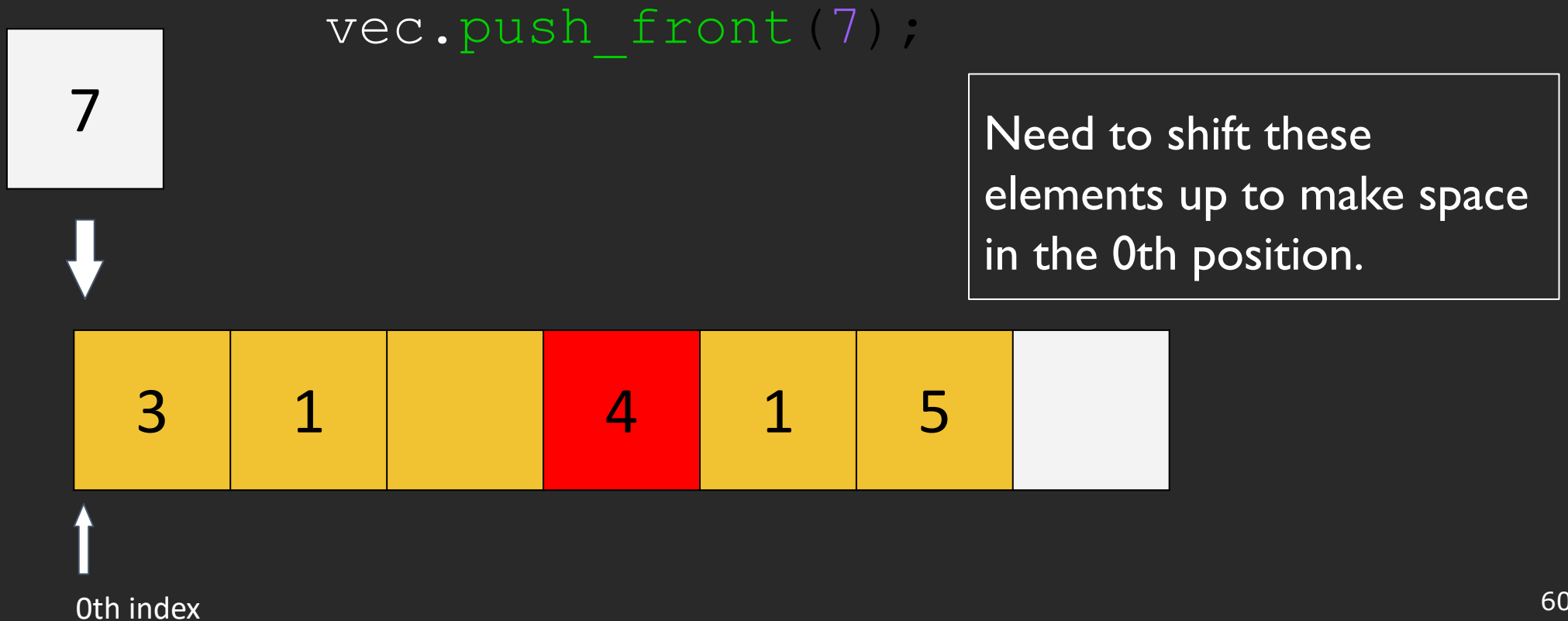
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

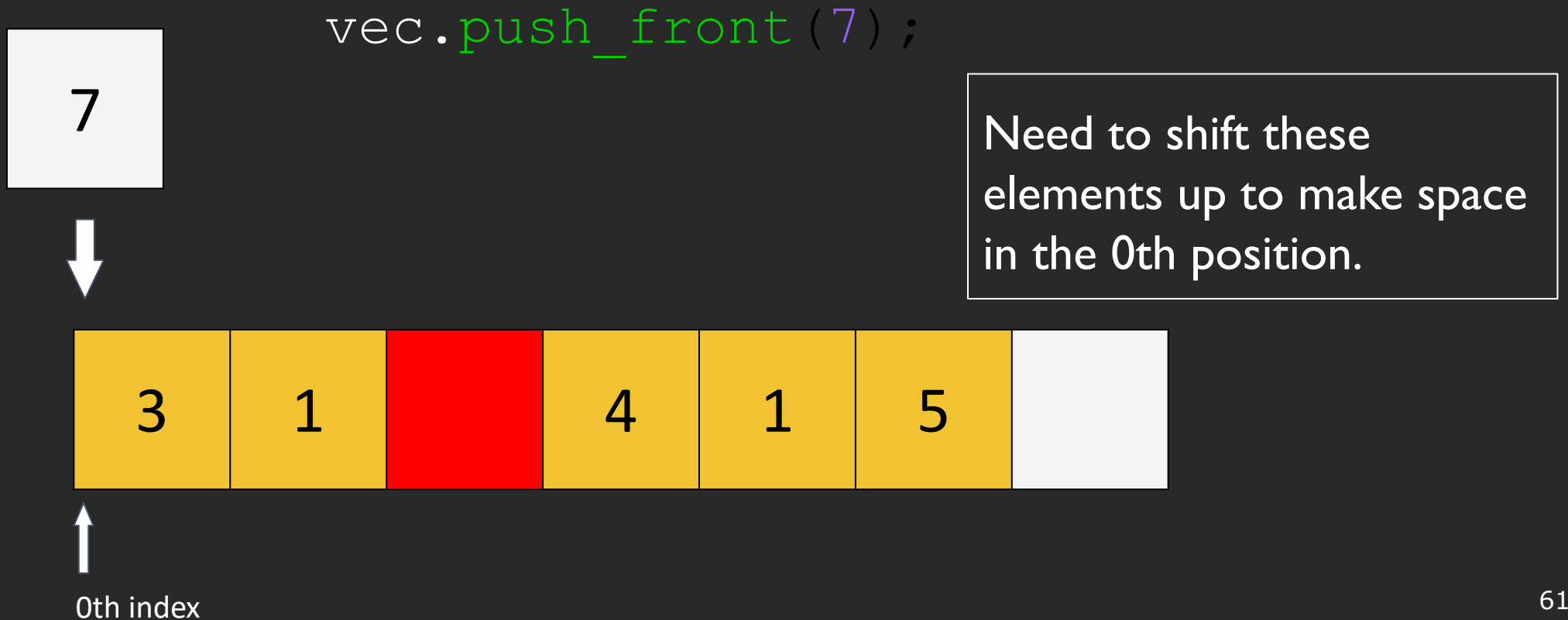
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

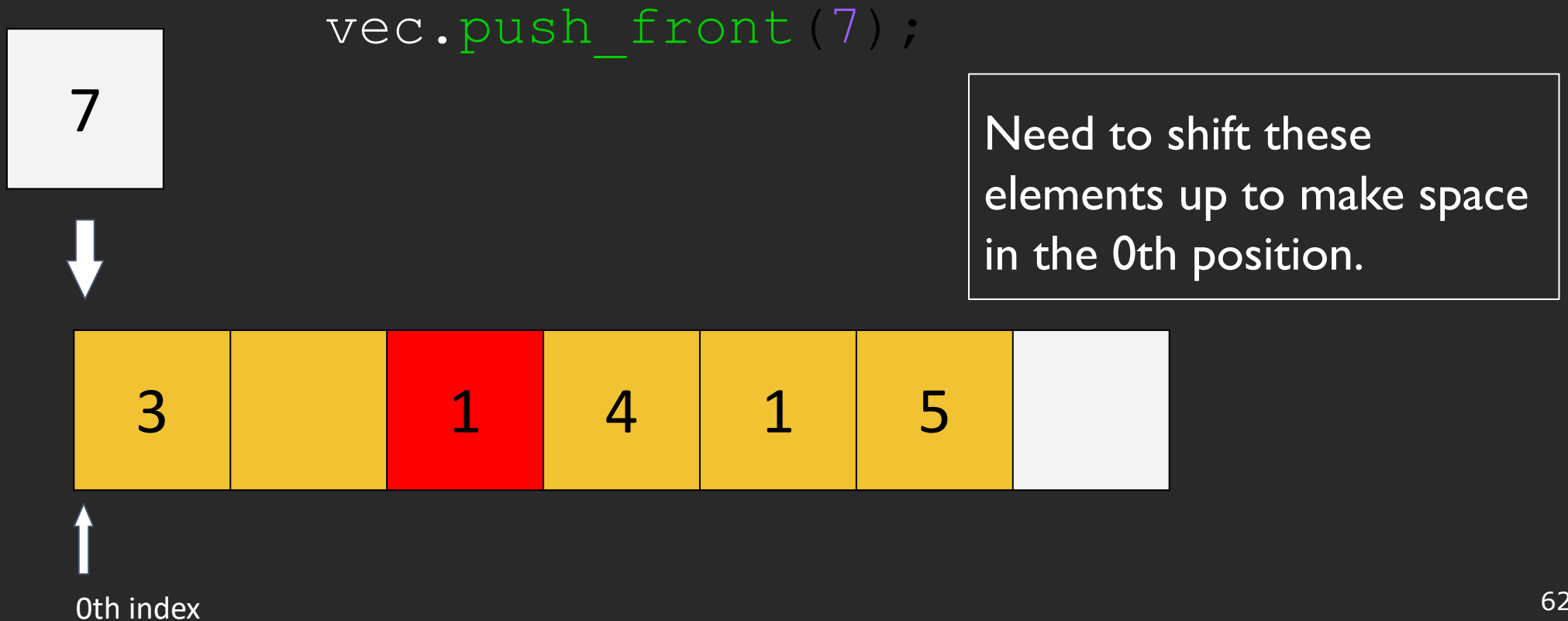
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

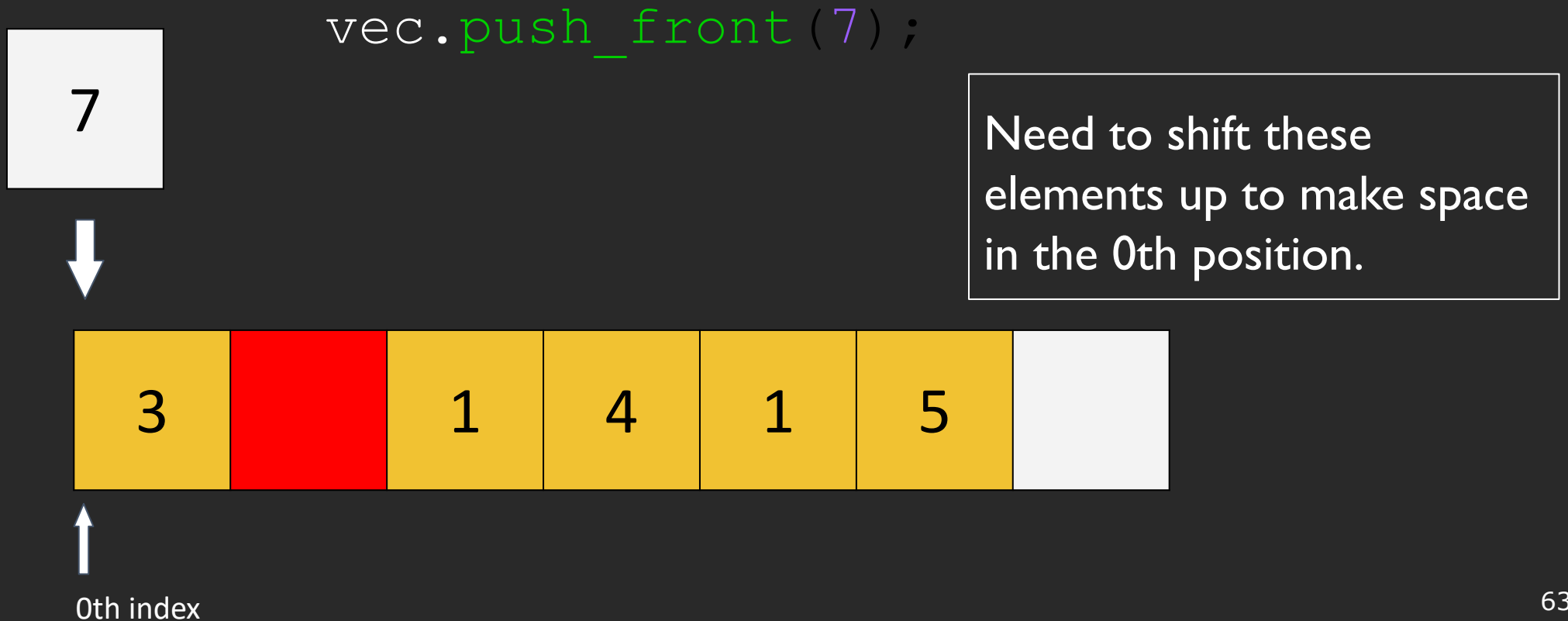
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.

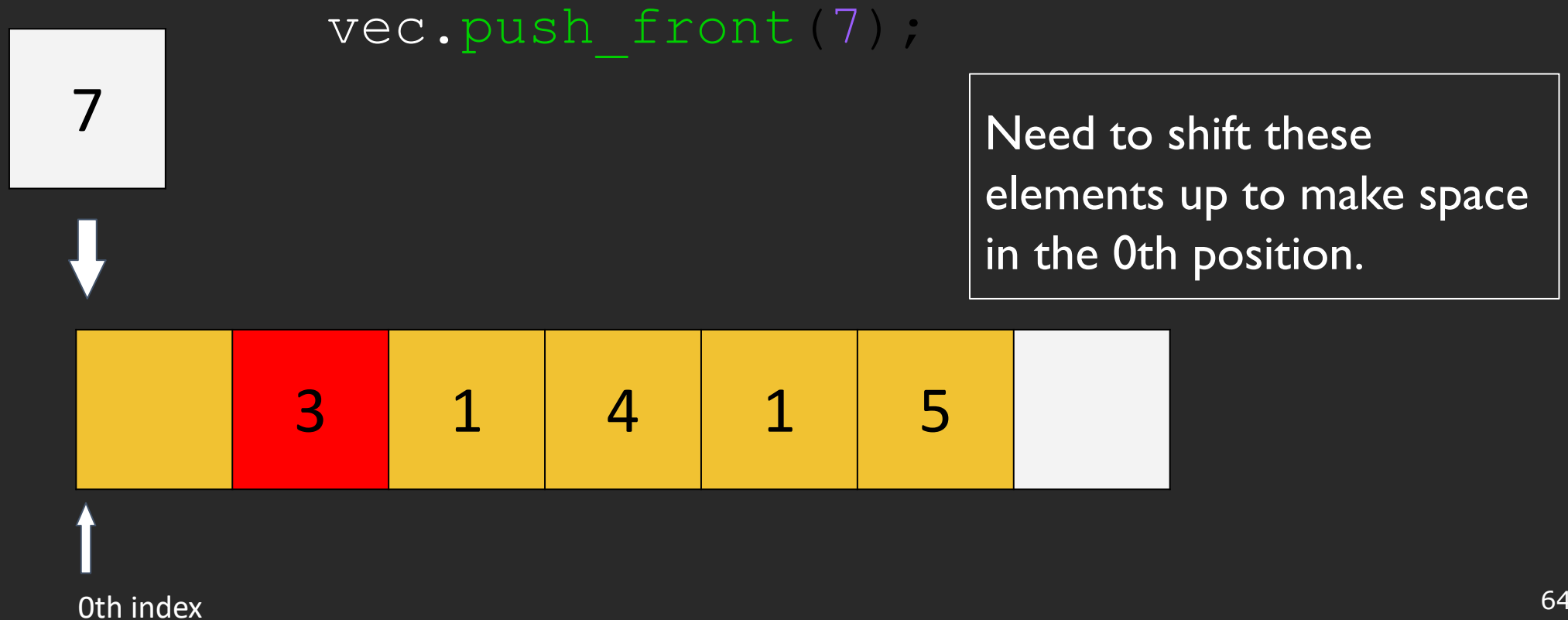
Let's look at a small vector:



Why is `push_front` slow?

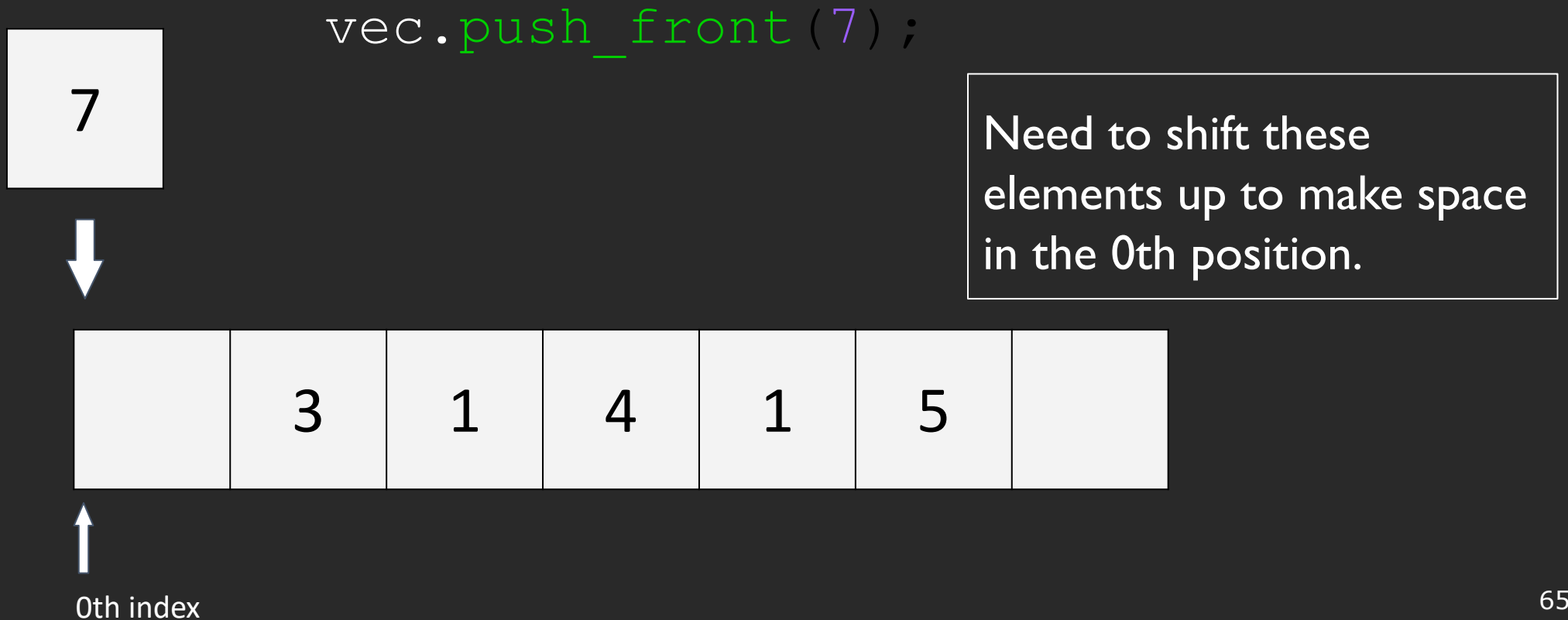
Suppose `push_front` existed and we used it.

Let's look at a small vector:



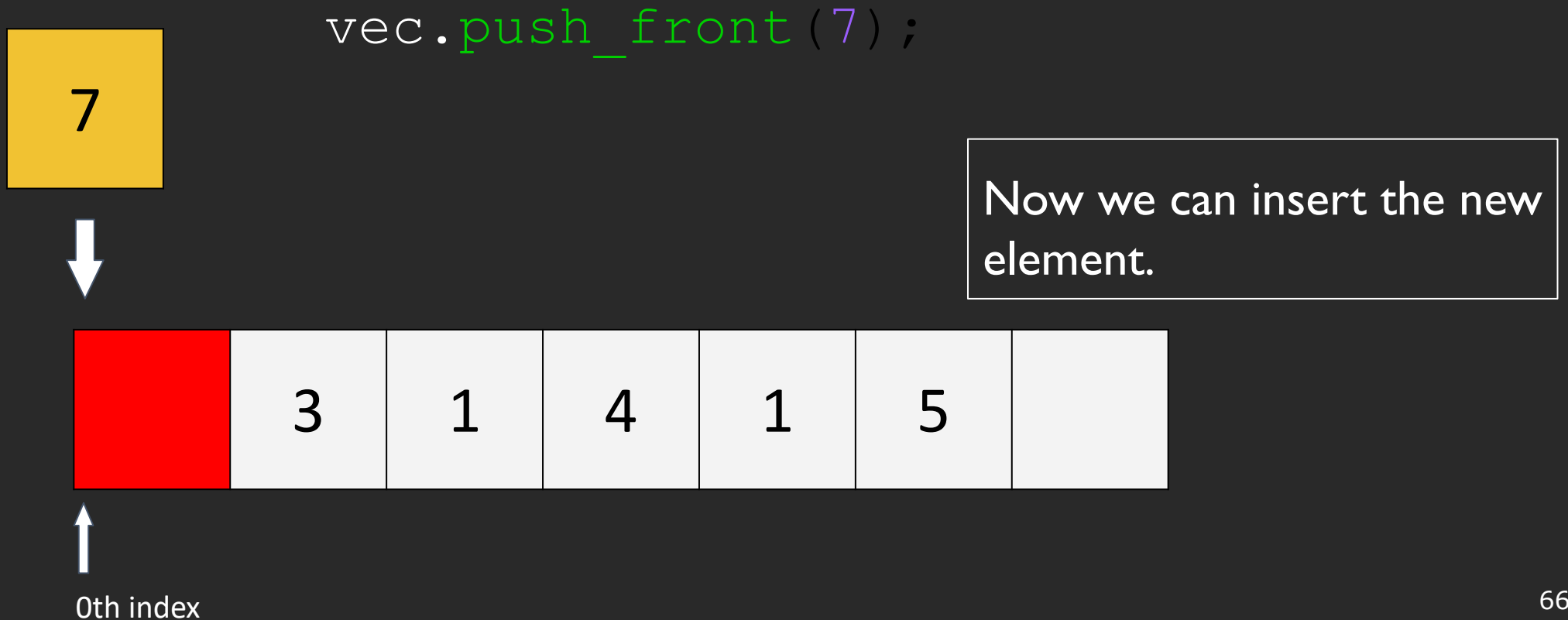
Why is `push_front` slow?

Suppose `push_front` existed and we used it.
Let's look at a small vector:



Why is `push_front` slow?

Suppose `push_front` existed and we used it.
Let's look at a small vector:

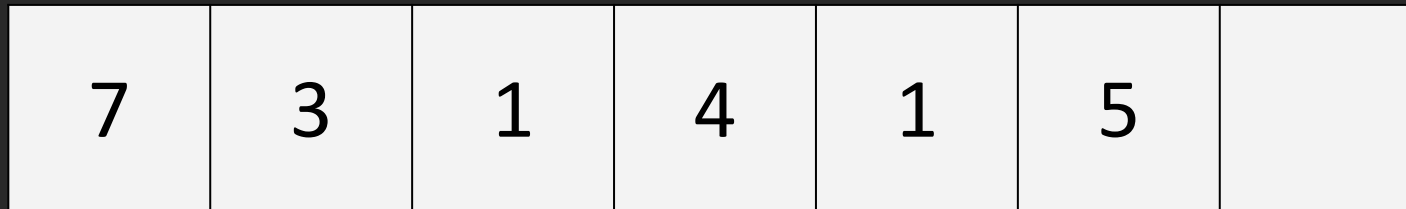


Why is `push_front` slow?

Suppose `push_front` existed and we used it.

Let's look at a small vector:

```
vec.push_front(7);
```



0th index

Why is `push_front` slow?



7	3	1	4	1	5	
---	---	---	---	---	---	--



0th index

Why is `push_front` slow?

A vector is the **prime** tool of choice in most applications!

- Fast
- Lightweight
- Intuitive

However, we just saw vectors only grow efficiently in **one direction**.

Sometimes it is useful to be able to `push_front` quickly!

C++ has a solution!

```
std::deque<T>
```

```
std::deque<T>
```

A deque (pronounced “deck”) is a **double ended queue**.

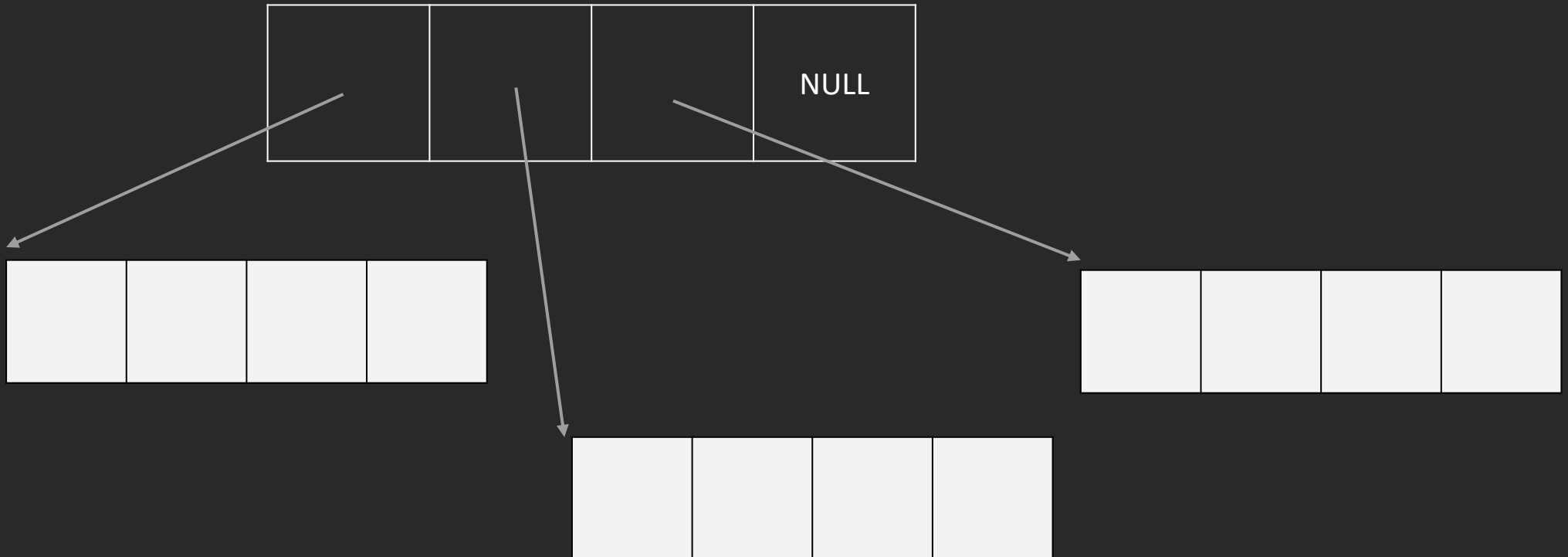
Can do **everything** a vector can do

and also...

Unlike a vector, it is possible (and **fast**) to `push_front` and `pop_front`.

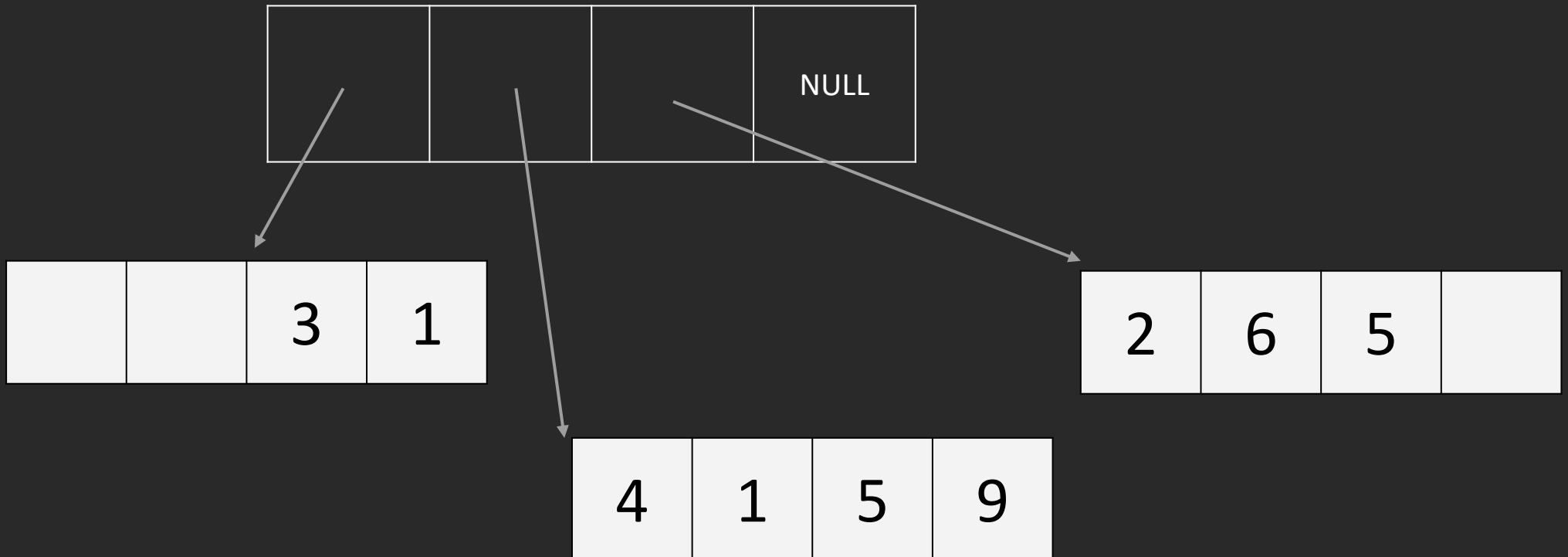
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



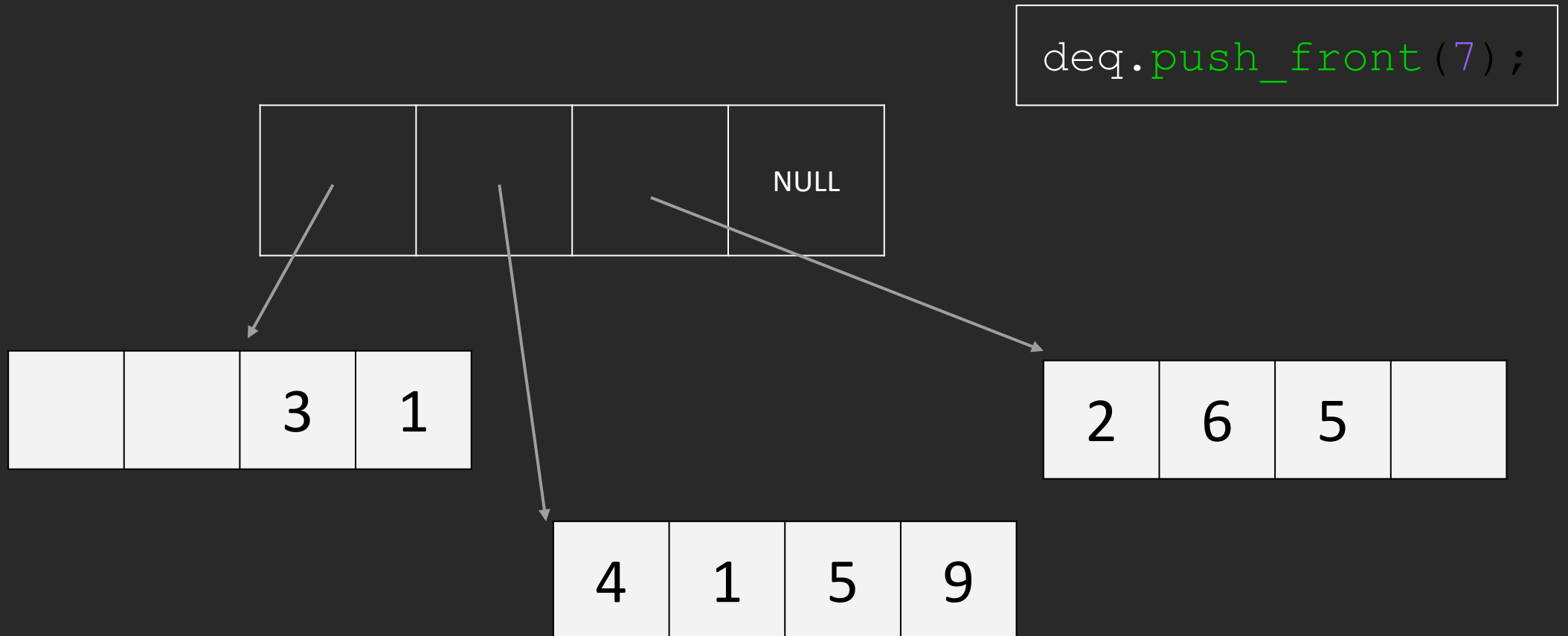
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



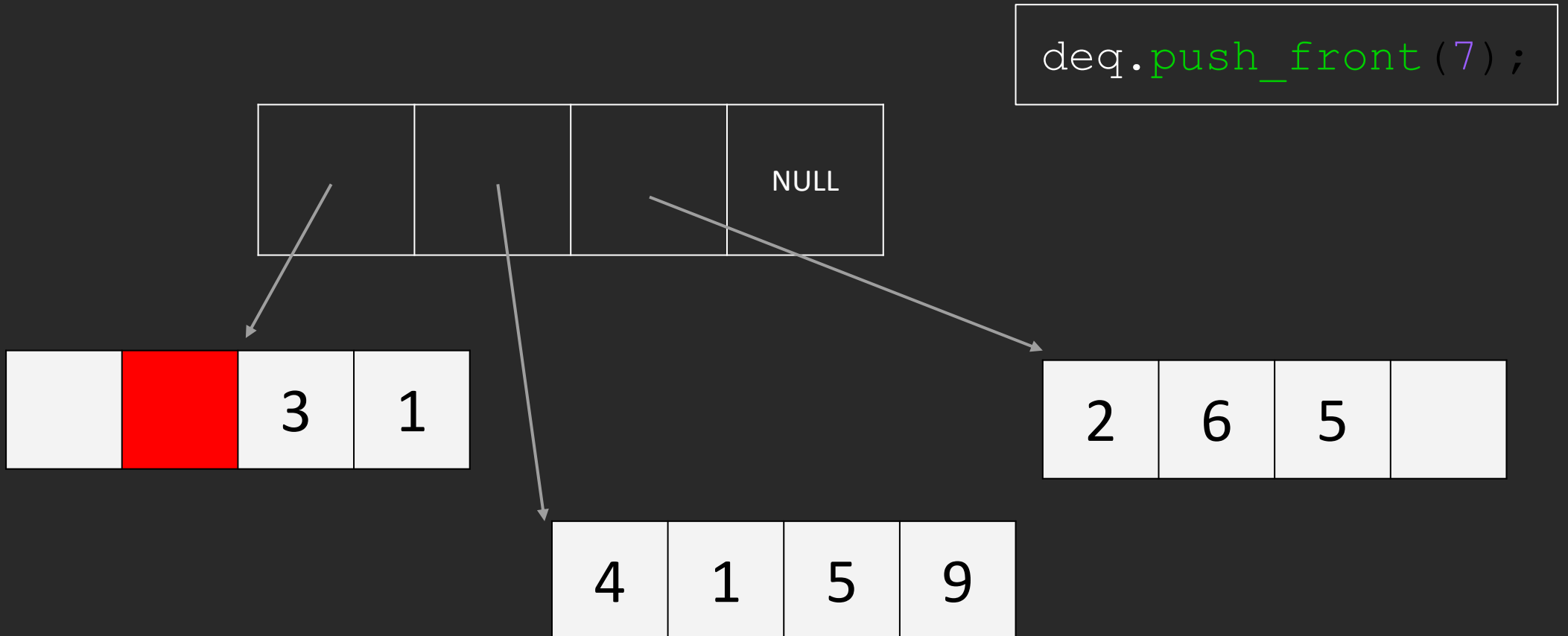
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



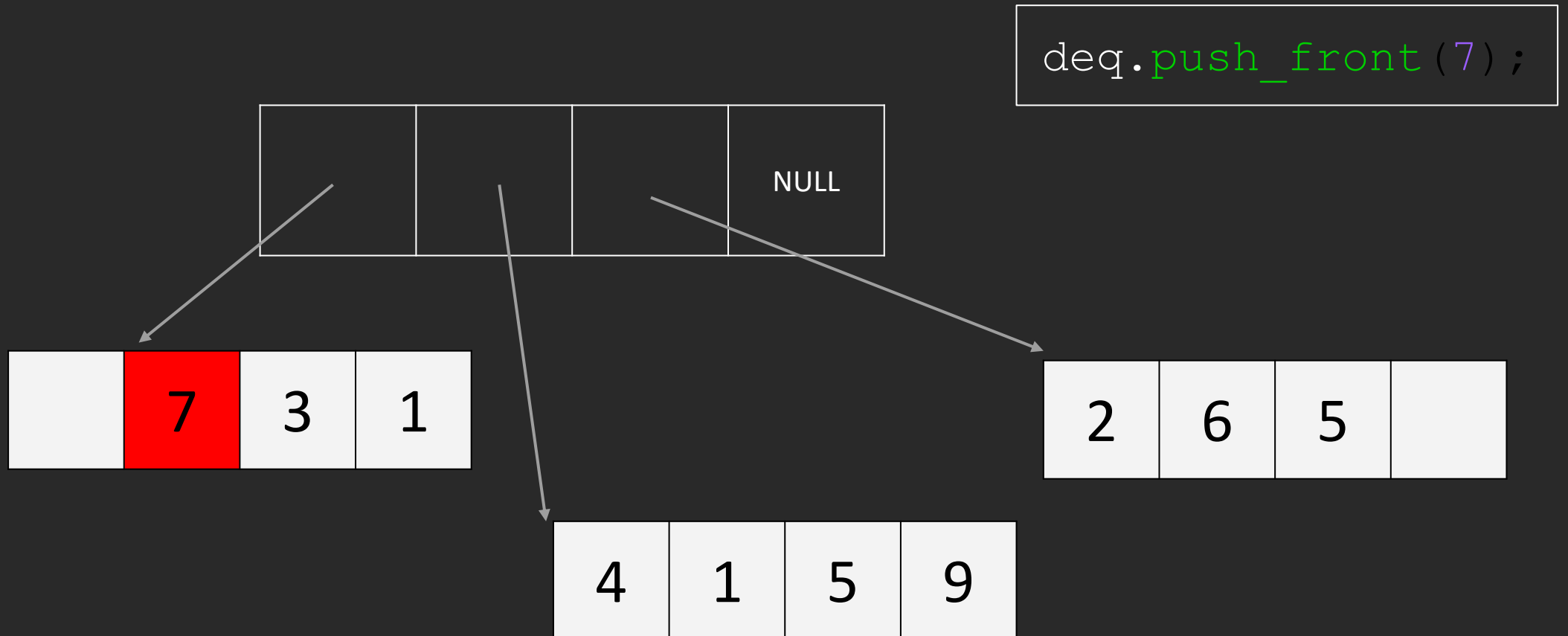
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



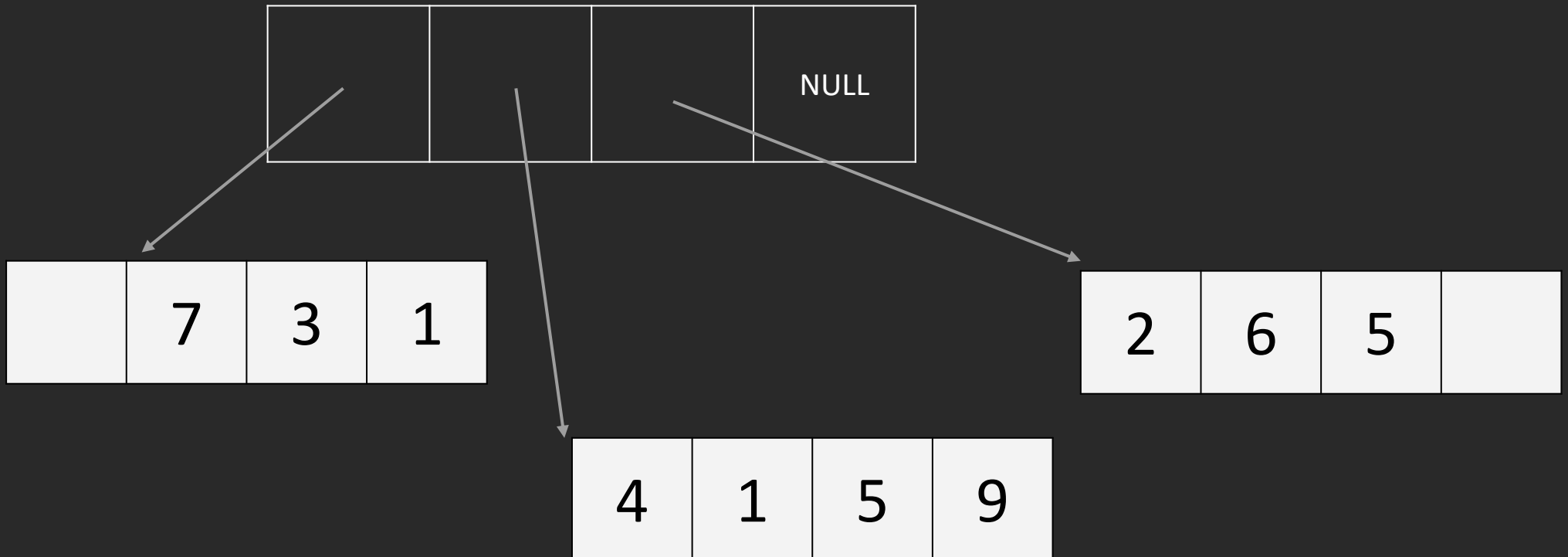
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



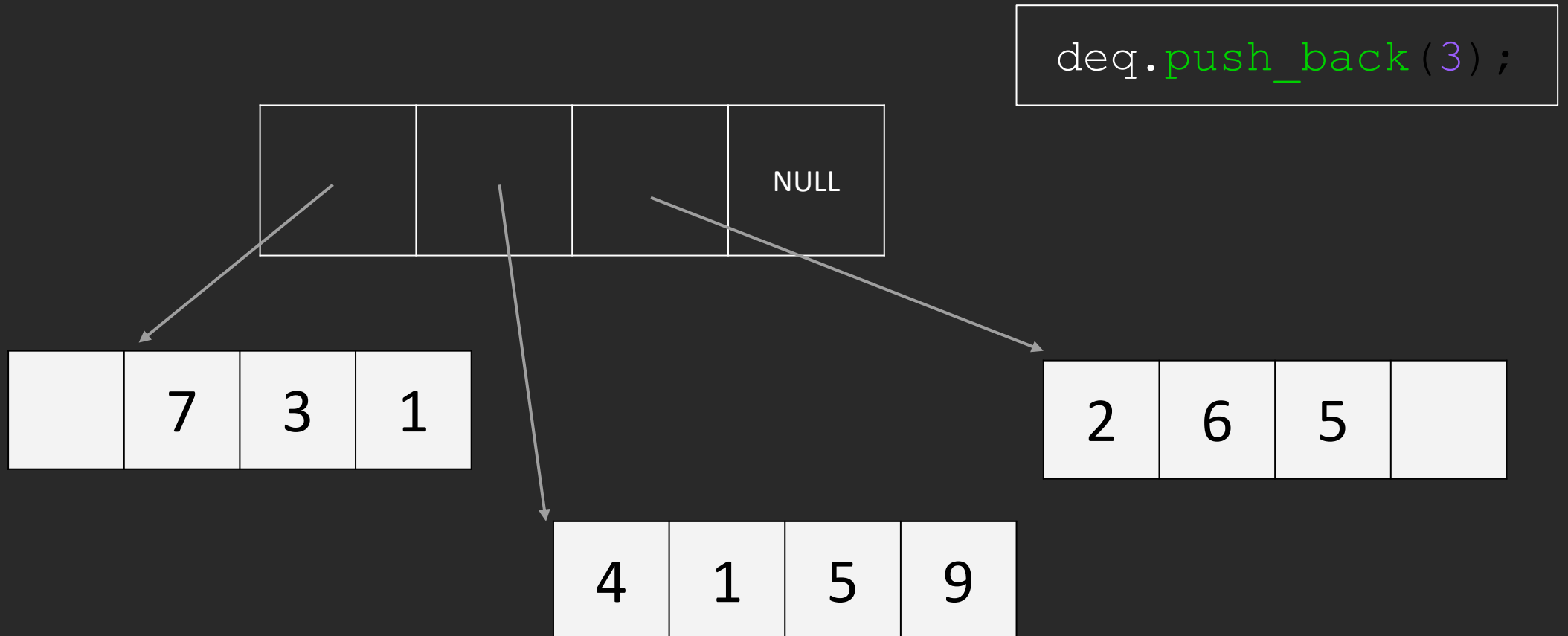
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



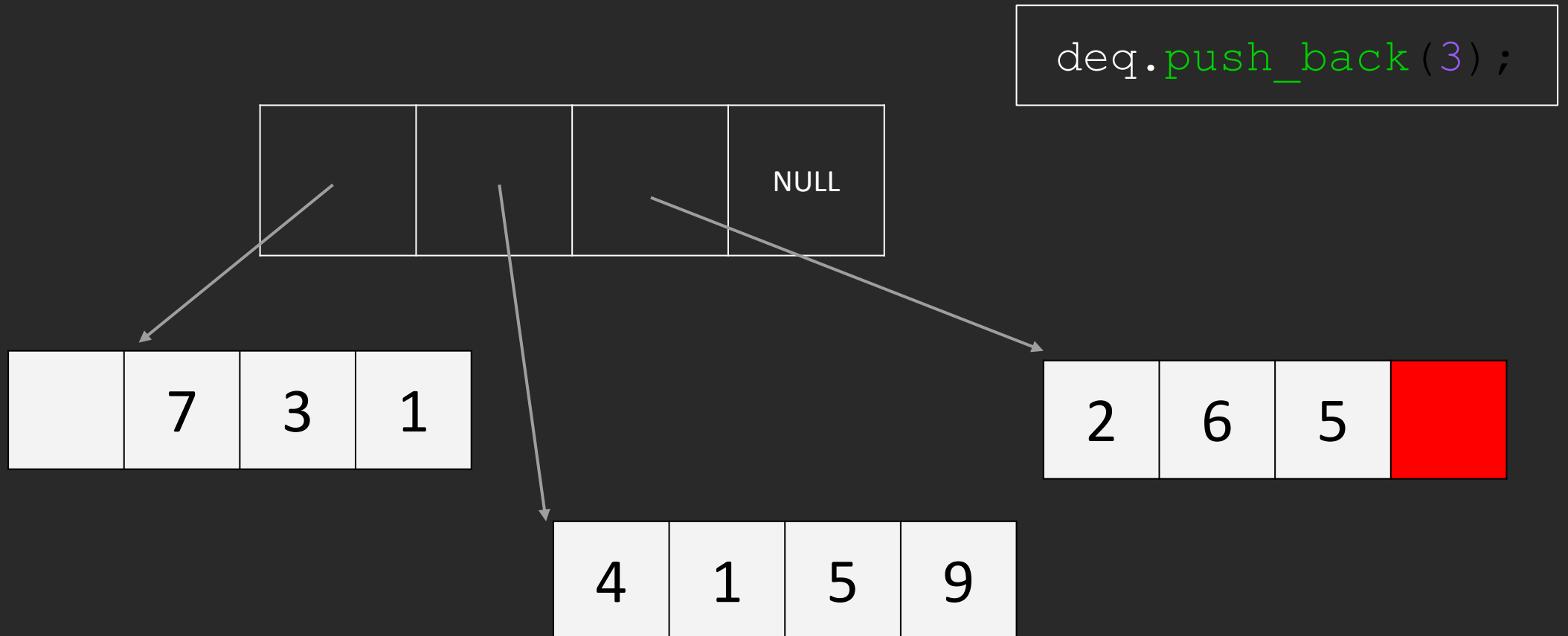
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



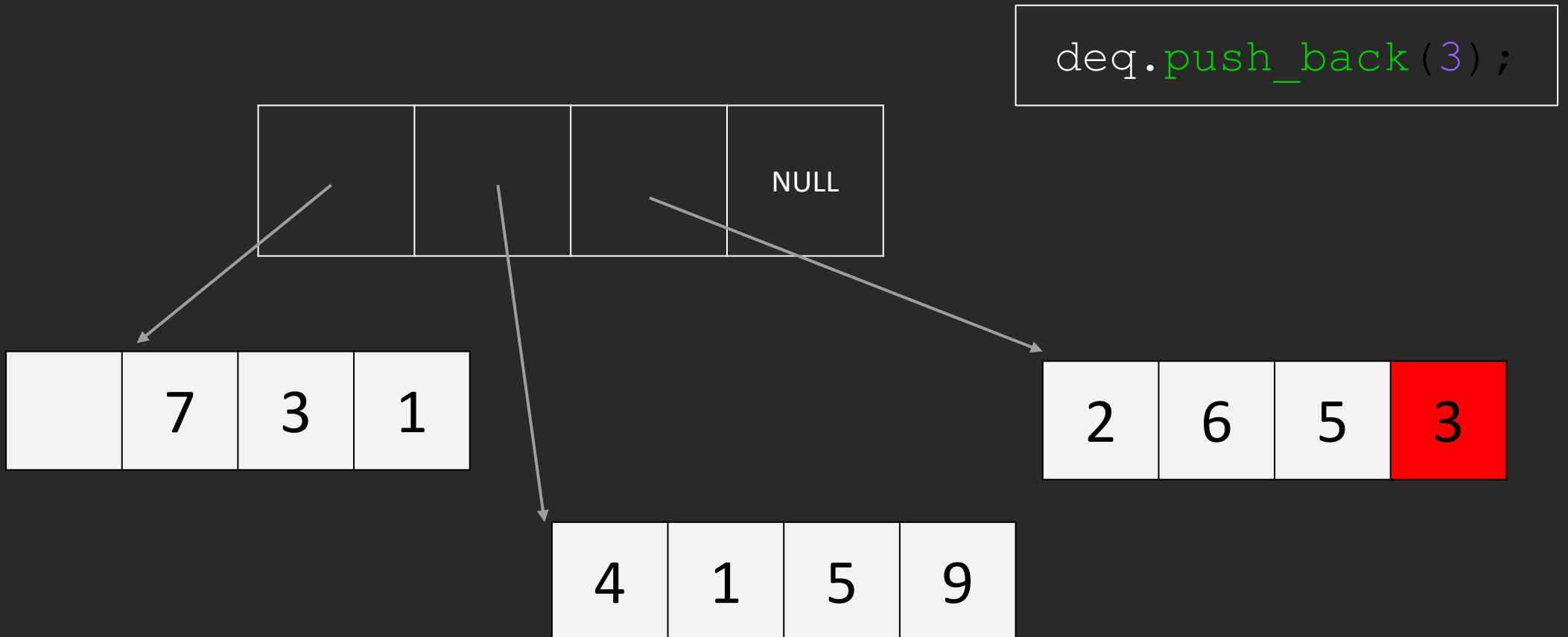
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



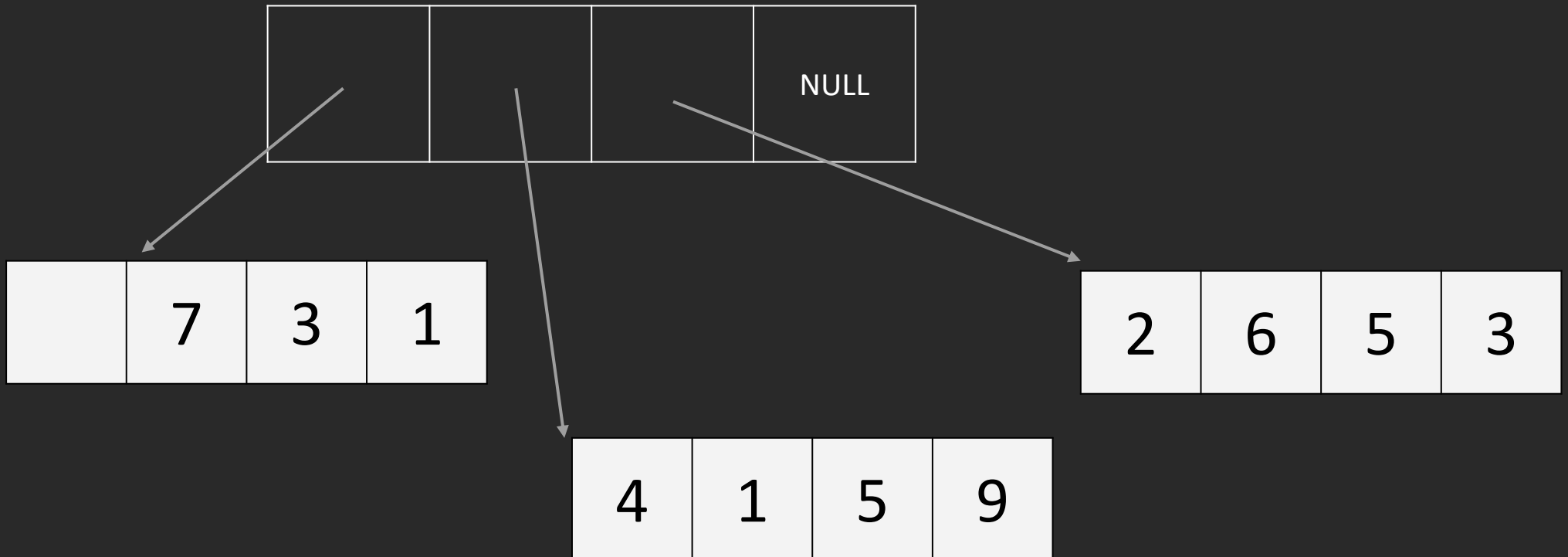
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



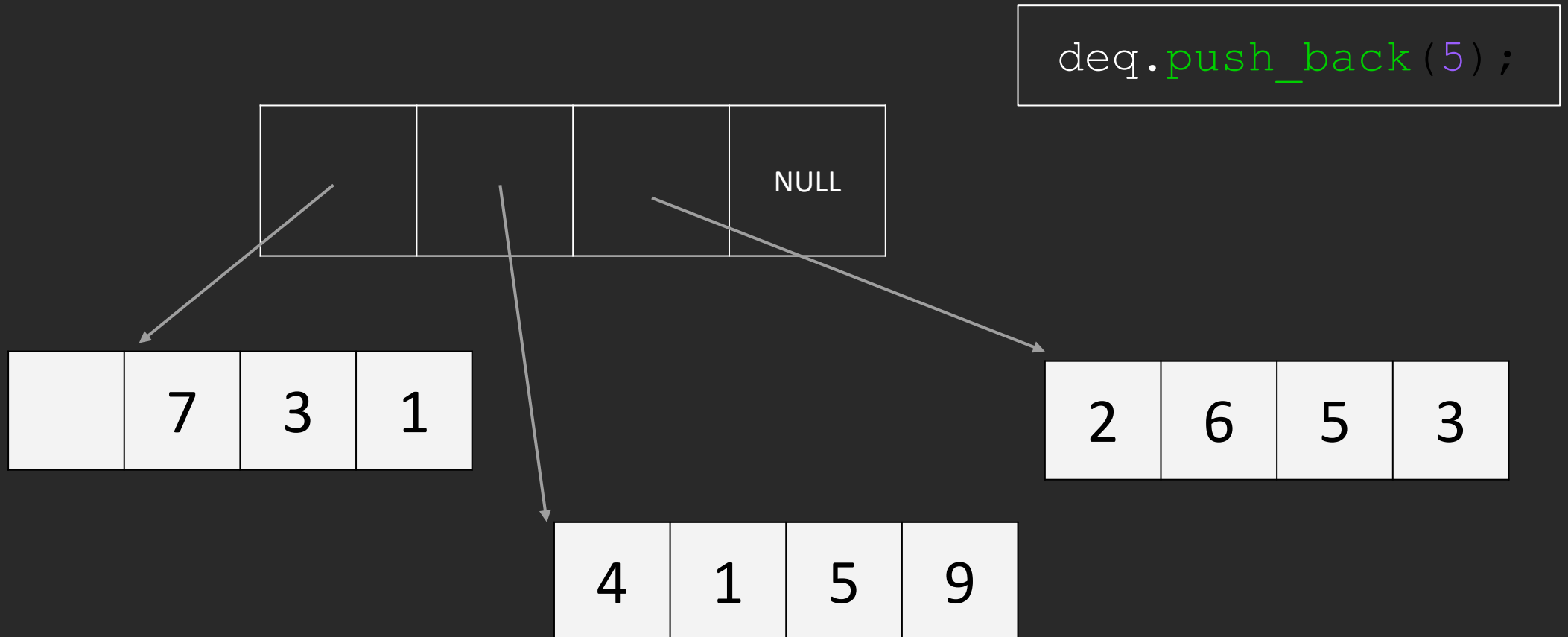
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



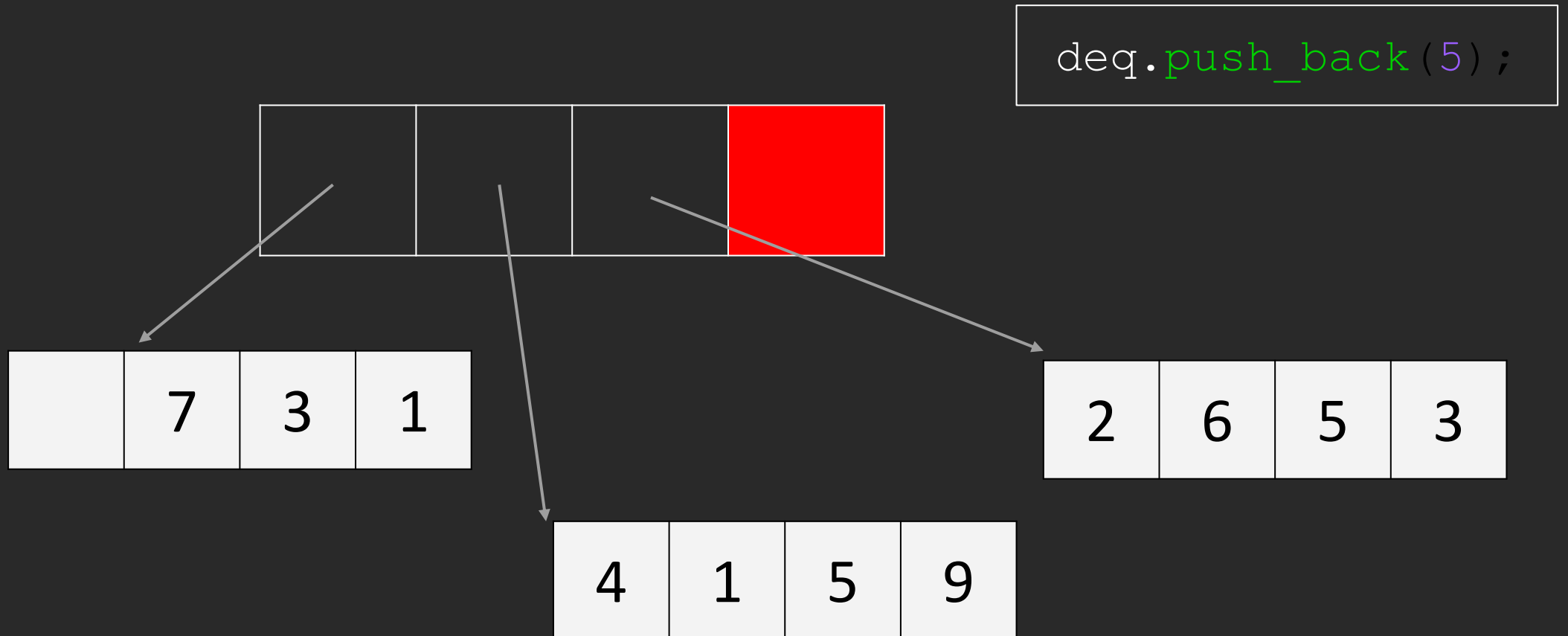
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



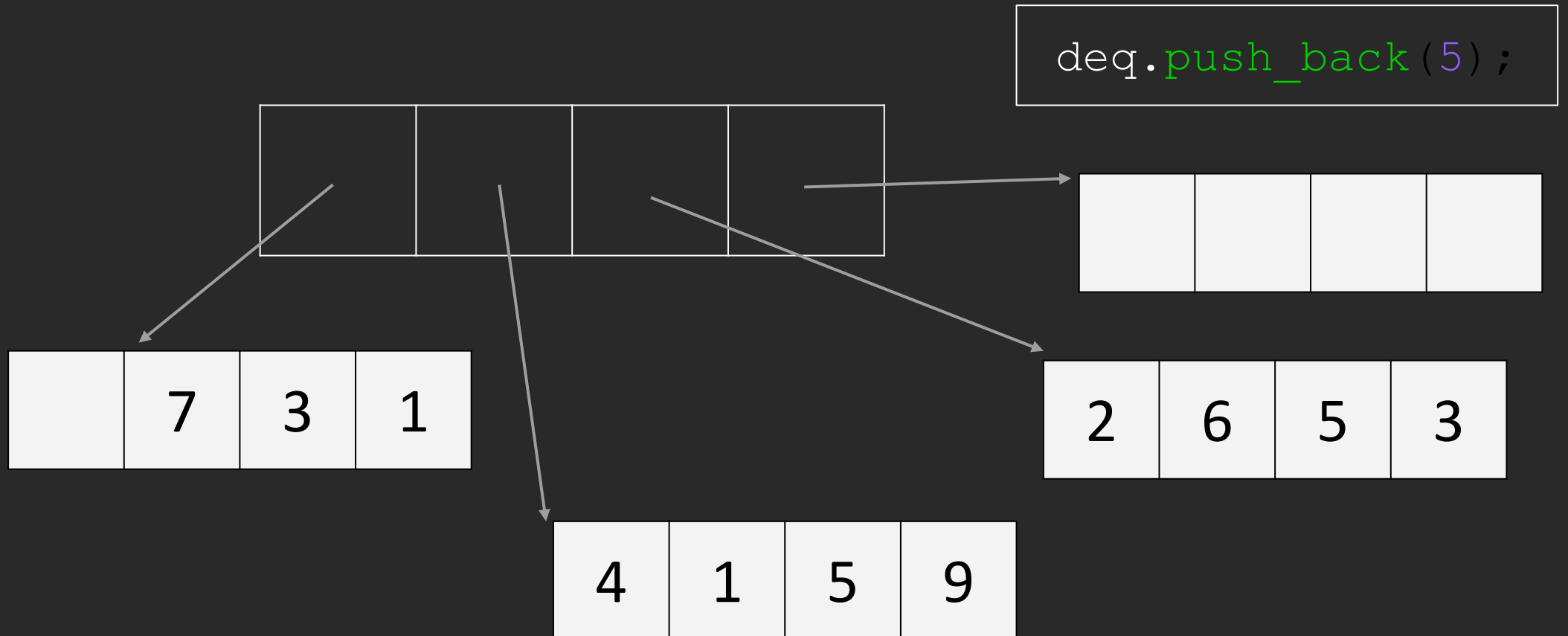
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



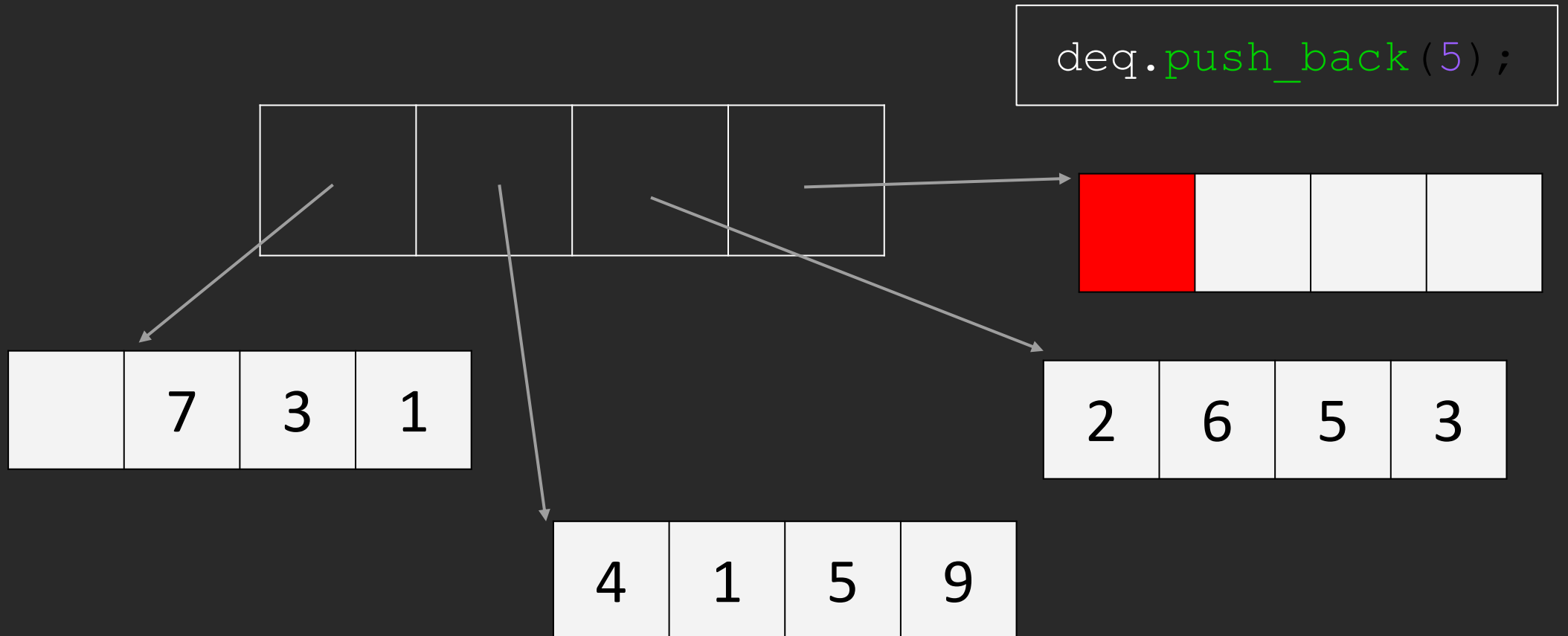
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



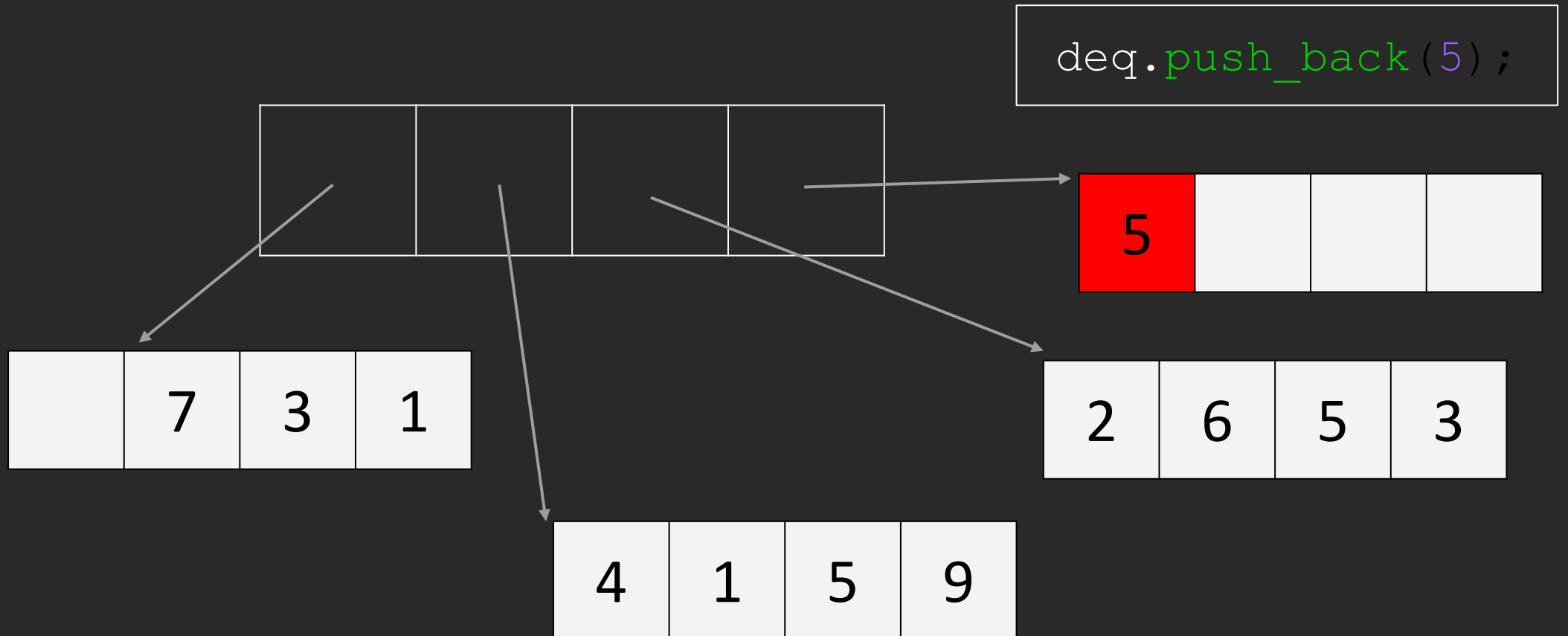
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



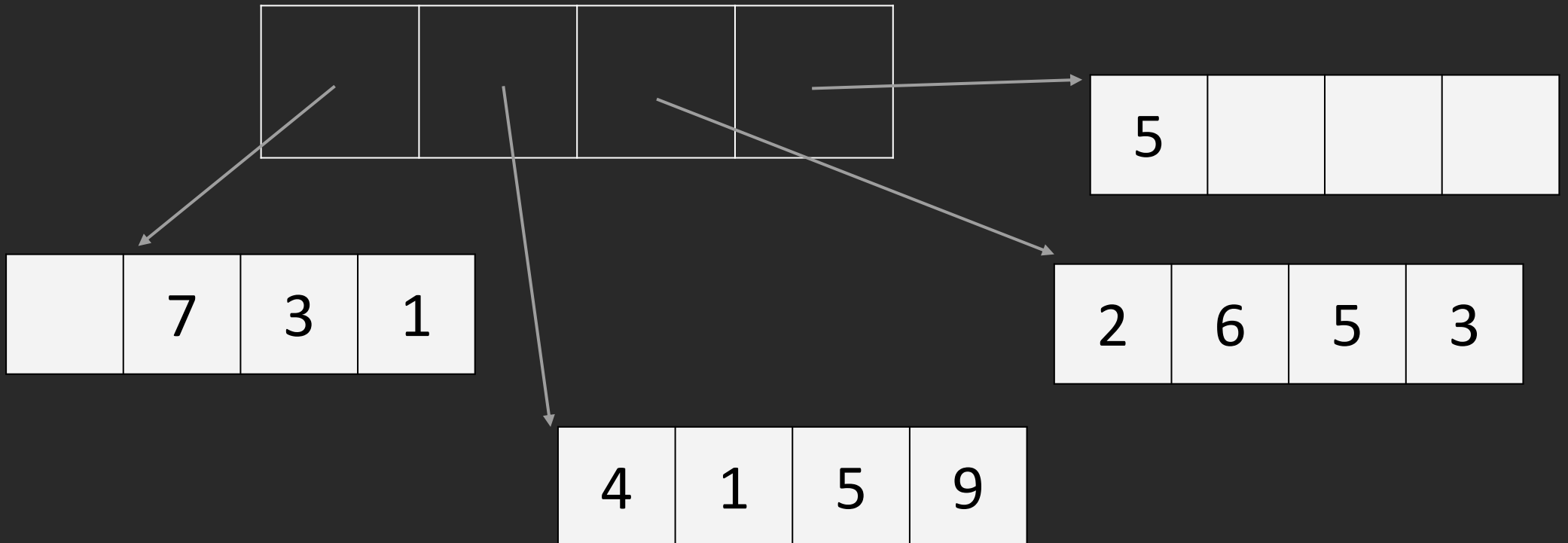
How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



How does `std::deque<T>` work?

There is no single specific implementation of a deque, but one common one might look like this:



Wait a minute...

Question

If deque can do **everything** a vector can do and **also** has a **fast** `push_front...`

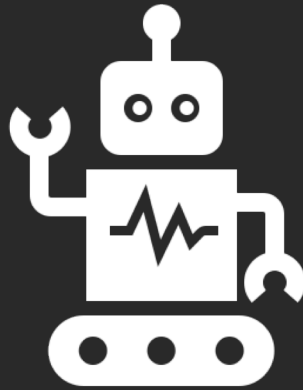
Why use a vector at all?

Downsides of `std::deque<T>`

Dequeues support **fast** `push_front` operations.

However, for other common operations like element access, vector will always outperform a deque.

Let's see this in action!



Example

Vector vs. Deque: Timing Version

(you'll use a similar concept with Assignment 1 as well!)

Which to Use?

*“vector is the type of sequence that should be used by **default**... deque is the data structure of choice when most insertions and deletions take place **at the beginning or at the end** of the sequence.”*

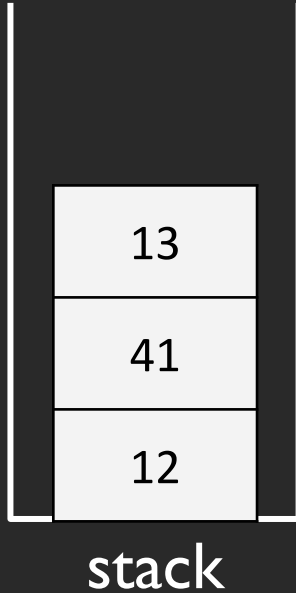
— C++ ISO Standard (section 23.1.1.2):



Container Adaptors

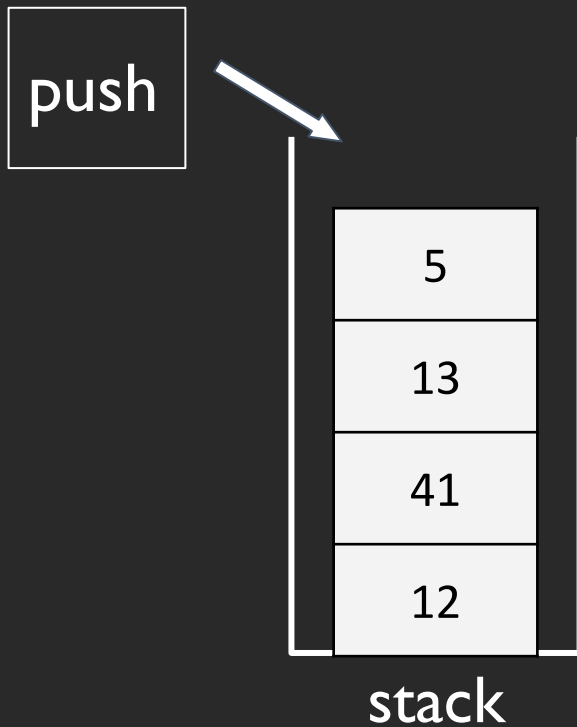
Container Adaptors

Recall stacks and queues:



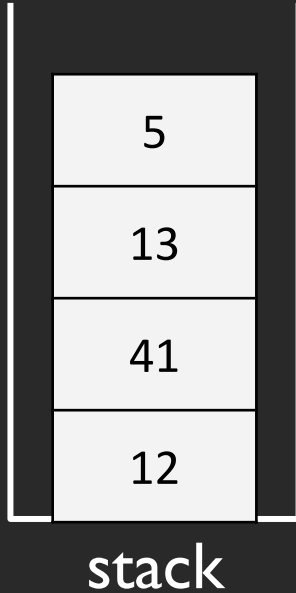
Container Adaptors

Recall stacks and queues:



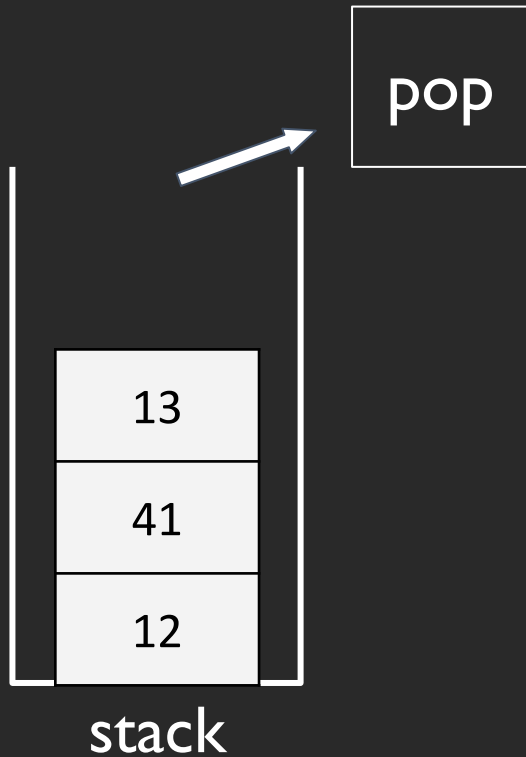
Container Adaptors

Recall stacks and queues:



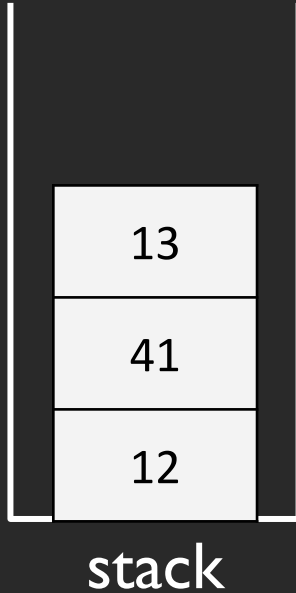
Container Adaptors

Recall stacks and queues:



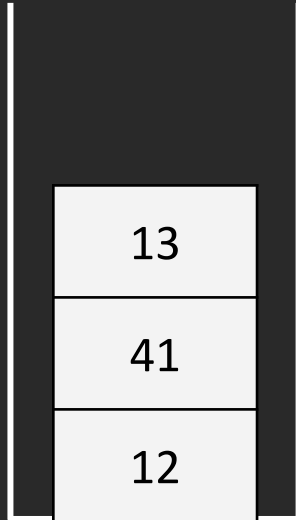
Container Adaptors

Recall stacks and queues:

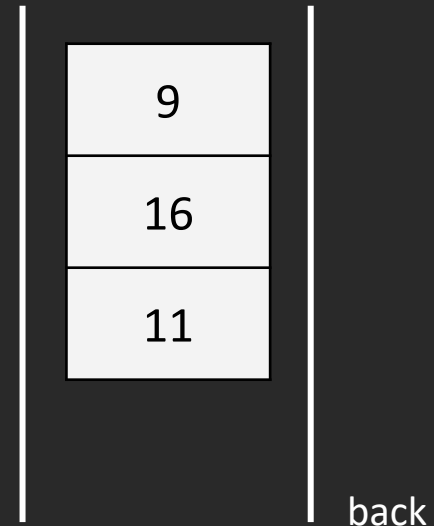


Container Adaptors

Recall stacks and queues:



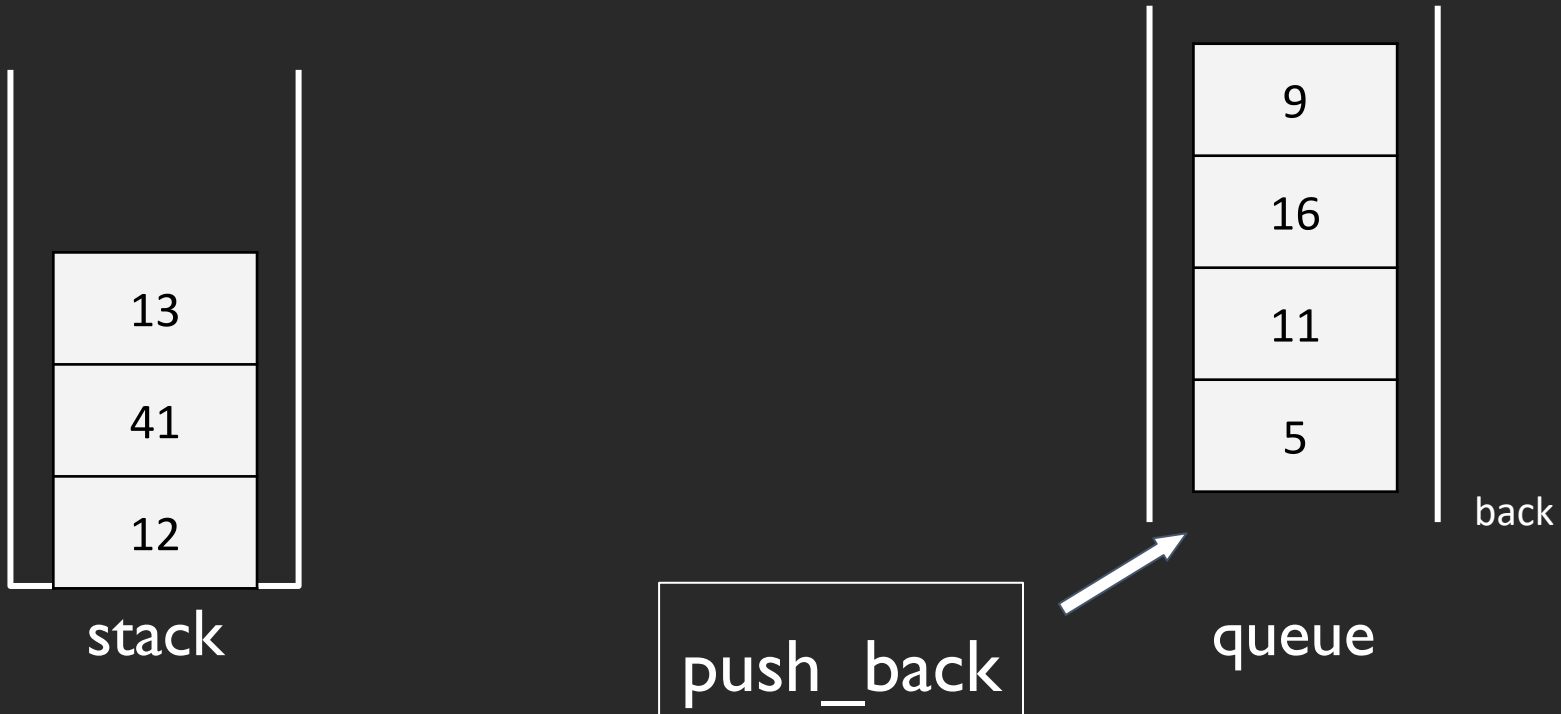
stack



queue

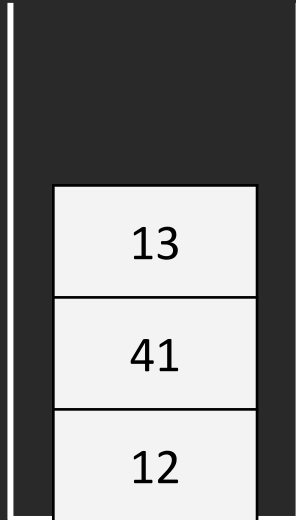
Container Adaptors

Recall stacks and queues:

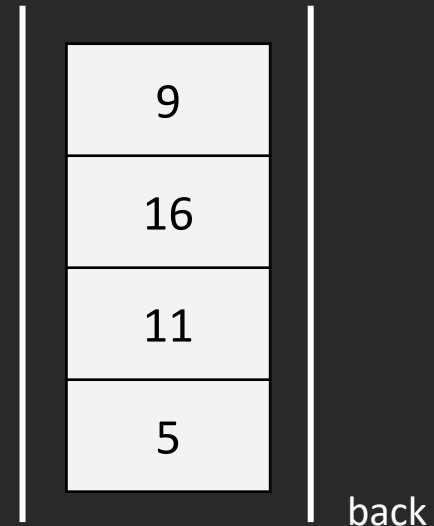


Container Adaptors

Recall stacks and queues:



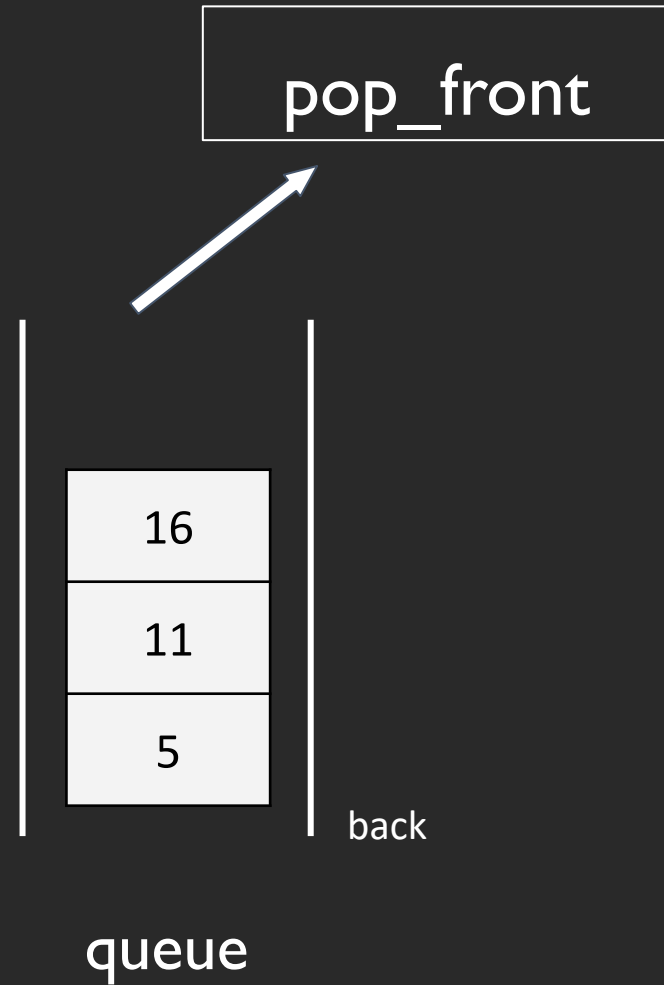
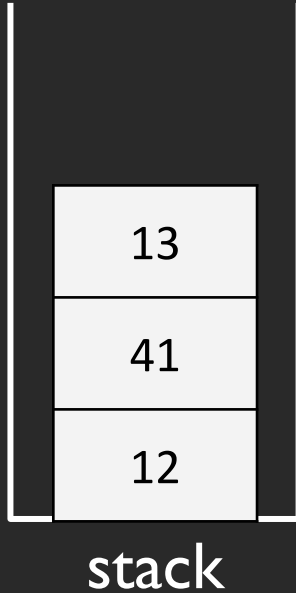
stack



queue

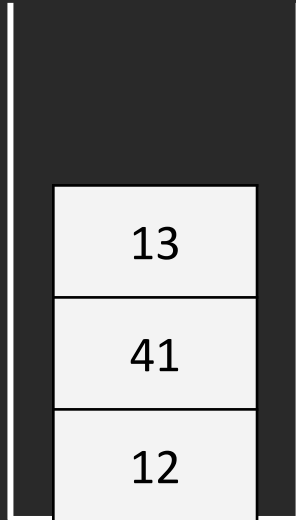
Container Adaptors

Recall stacks and queues:

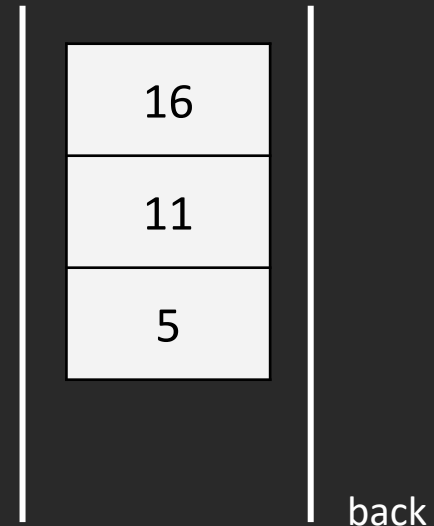


Container Adaptors

Recall stacks and queues:



stack



queue

Container Adaptors

How can we implement stack and queue using the containers we have?

Container Adaptors

How can we implement stack and queue using the containers we have?

Stack:

Just limit the functionality of a vector/deque to only allow `push_back` and `pop_back`.

Queue:

Just limit the functionality of a deque to only allow `push_back` and `pop_front`.

Plus only allow access to **top** element

Container Adaptors

For this reason, stacks and queues are known as **container adaptors**.

std::stack

Defined in header `<stack>`

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

The `std::stack` class is a container adapter that gives the programmer the functionality of a stack - specifically, a FILO (first-in, last-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The stack pushes and pops the element from the back of the underlying container, known as the top of the stack.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)
- Container** - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:
- `back()`
 - `push_back()`
 - `pop_back()`
- The standard containers `std::vector`, `std::deque` and `std::list` satisfy these requirements.

std::queue

Defined in header `<queue>`

```
template<
    class T,
    class Container = std::deque<T>
> class queue;
```

The `std::queue` class is a container adapter that gives the programmer the functionality of a queue - specifically, a FIFO (first-in, first-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The queue pushes the elements on the back of the underlying container and pops them from the front.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)
- Container** - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:
- `back()`
 - `front()`
 - `push_back()`
 - `pop_front()`
- The standard containers `std::deque` and `std::list` satisfy these requirements.

Container Adaptors

For this reason, stacks and queues are known as **container adaptors**.

std::stack

Defined in header `<stack>`

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

The `std::stack` class is a container adapter that gives the programmer the functionality of a stack - specifically, a FILO (first-in, last-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The stack pushes and pops the element from the back of the underlying container, known as the top of the stack.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)
- Container** - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:
- `back()`
 - `push_back()`
 - `pop_back()`
- The standard containers `std::vector`, `std::deque` and `std::list` satisfy these requirements.

std::queue

Defined in header `<queue>`

```
template<
    class T,
    class Container = std::deque<T>
> class queue;
```

The `std::queue` class is a container adapter that gives the programmer the functionality of a queue - specifically, a FIFO (first-in, first-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The queue pushes the elements on the back of the underlying container and pops them from the front.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)
- Container** - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:
- `back()`
 - `front()`
 - `push_back()`
 - `pop_front()`
- The standard containers `std::deque` and `std::list` satisfy these requirements.

Container Adaptors

For this reason, stacks and queues are known as **container adaptors**.

std::stack

Defined in header `<stack>`

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

The `std::stack` class is a container adapter that gives the programmer the functionality of a stack - specifically, a FILO (first-in, last-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The stack pushes and pops the element from the back of the underlying container, known as the top of the stack.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)

Container - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:

- `back()`
- `push_back()`
- `pop_back()`

The standard containers `std::vector`, `std::deque` and `std::list` satisfy these requirements.

std::queue

Defined in header `<queue>`

```
template<
    class T,
    class Container = std::deque<T>
> class queue;
```

The `std::queue` class is a container adapter that gives the programmer the functionality of a queue - specifically, a FIFO (first-in, first-out) data structure.

The class template acts as a wrapper to the underlying container - only a specific set of functions is provided. The queue pushes the elements on the back of the underlying container and pops them from the front.

Template parameters

- T** - The type of the stored elements. The behavior is undefined if T is not the same type as `Container::value_type`. (since C++17)

Container - The type of the underlying container to use to store the elements. The container must satisfy the requirements of [SequenceContainer](#). Additionally, it must provide the following functions with the usual semantics:

- `back()`
- `front()`
- `push_back()`
- `pop_front()`

The standard containers `std::deque` and `std::list` satisfy these requirements.

Preview of Assignment 1

due Thursday, October 17

overflow

file streams

Review: read file line by line

```
ifstream file(filename); // open
string line;

while (getline(file, line)) {
    // do something with line
}

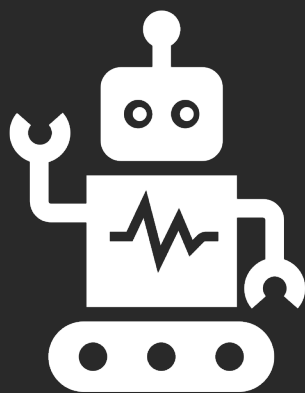
file.close(); // technically don't need this
```

Review: read file token by token

```
ifstream file(filename); // open
string token;

while (file >> token) {
    // do something with line
}

file.close(); // technically don't need this
```



Example

implementing Axxess, take 2

manipulators +
overloading >> or <<

There are some keywords that will change the behavior of the stream when inserted.

<code>endl</code>	insert newline and flush stream
<code>ws</code>	skips all whitespace until it finds another char
<code>boolalpha</code>	prints "true" and "false" for bools.
<code>hex</code>	prints numbers in hexadecimal
<code>setprecision</code>	adjusts the precision of printed numbers

We can use manipulators to pad the output.

```
cout << "[" << setw(10) << "Ito" << "];
```

Output: [Hi]

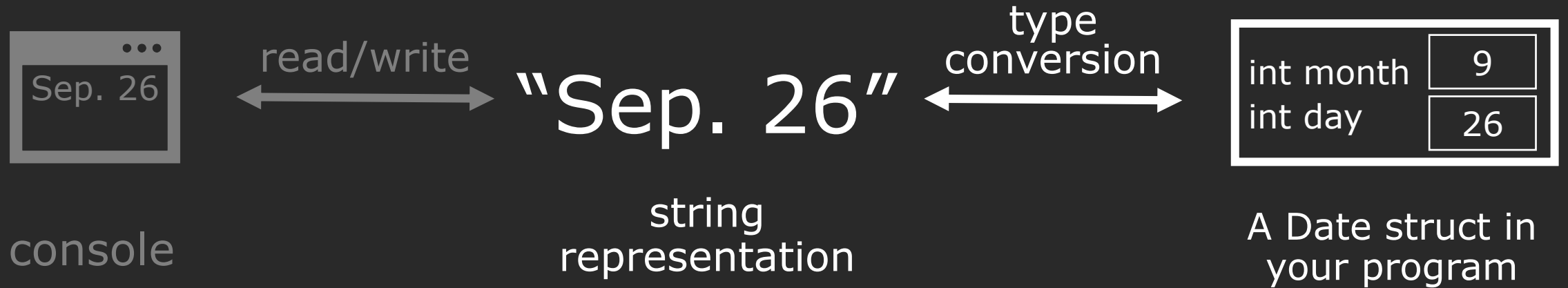
```
cout << "[" << left << setw(10) << "Ito" << "];
```

Output: [Hi]

```
cout << "[" << left << setfill('-') << setw(10) << "Ito" << "];
```

Output: [Hi-----]

C++ does not know how to convert a custom structure to/from a string.



You have to specify how the conversion to/from a string occurs.

```
ostream& operator<<(ostream& os, const Time& time) {  
    os << time.hour << ":"  
        << setfill('0') << setw(2) << time.minute;  
    return os;  
}
```

Other programming languages also allow you to turn an object into a string.

```
// java
String toString() {
    // create string from this object
}
```

```
// python
def __str__(self):
    // create string from self
```

You have to specify how the conversion to/from a string occurs.

```
ostream& operator<<(ostream& os, const Time& time) {  
    os << time.hour << ":"  
        << setfill('0') << setw(2) << time.minute;  
    return os;  
}
```

[ostream object] << [Time struct];

The header literally means:
how should this be interpreted.

Make sure to respect the return value of the << operator.

```
ostream& operator<<(ostream& os, const Time& time) {  
    os << time.hour << ":"  
        << setfill('0') << setw(2) << time.minute;  
    return os;  
}
```

```
[ostream] << [Time] << [Time];
```

The return value is what allows us to chain the <<'s.

You have to specify how the conversion to/from a string occurs.

```
ostream& operator<<(ostream& os, const Time& time) {  
    os << time.hour << ":"  
        << setfill('0') << setw(2) << time.minute;  
    return os;  
}
```

"13:04"

string
representation

type
conversion
←

int hour	13
int minute	4

A Time struct in
your program

Manipulators can help you format the string exactly as you want it.

```
ostream& operator<<(ostream& os, const Time& time) {  
    os << time.hour << ":"  
        << setfill('0') << setw(2) << time.minute;  
    return os;  
}
```

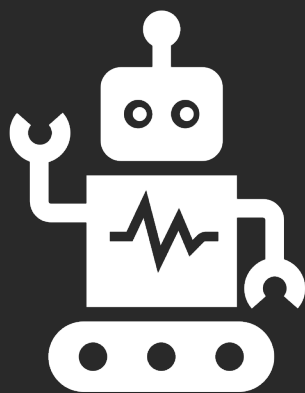
"13:04"

string
representation

type
conversion
←

int hour	13
int minute	4

A Time struct in
your program



Example

implementing `Axess`, `stream >> overload`

Reading input is harder because you have to worry about error-checking.

```
istream& operator>>(istream& is, Time& time) {  
    ???  
    return is;  
}
```

"13:04"

string
representation

type
conversion
→

int hour	13
int minute	4

A Time struct in
your program

Be consistent with other input stream operations!

1. If the fail bit is on, do not do anything.
2. You can only read one token, nothing more, nothing less. (probably a good idea to copy the token into a stringstream).
3. If the operation failed, set the fail bit. Make sure the original stream and object are unchanged!

Be consistent with other input stream operations!

1. If the fail bit is on, do not do anything.

```
istream& operator>>(istream& is, Time& time) {  
    if (!is) return is; // if fail do nothing  
  
    // continued  
  
}
```

Be consistent with other input stream operations!

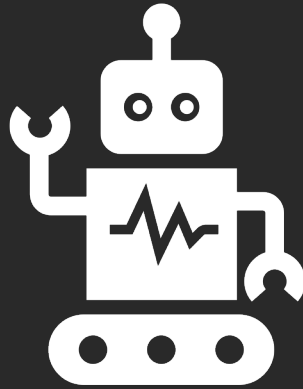
2. You can only read one token, nothing more, nothing less.
(probably a good idea to copy the token into a stringstream).

```
string timeString;  
if (!(is >> timeString)) { // read exactly one token  
    is.setstate(ios::failbit);  
    return is;  
}  
stringstream ss(timeString);  
// use ss;
```

Be consistent with other input stream operations!

3. If the operation failed, set the fail bit. Make sure the original stream and object are unchanged!

```
int hour, minute; char colon;
if (/* parse hour, minute, colon using >> */) {
    time = Time{hour, minute};
} else { // we didn't change time
    is.setstate(ios::failbit);
}
```



Example

implementing Axxess, take 2